Chapter 1
Introduction to C

CSE 4107 : Structured Programming I
REYNEP

1.1.1. History of C : Origins

C is a by-product of the UNIX. UNIX was 1 of the first Operating Systems, developed at Bell Laboratories by Ken Thompson, Dennis Ritchie, and others. Thompson single-handedly wrote the original version of UNIX, which ran on the DEC PDP-7 computer, an early minicomputer with only 8K words of main memory (this was 1969, after all!).

Like other operating systems of the time, UNIX was written in assembly language. Programs written in assembly language are usually painful to debug and hard to enhance; UNIX was no exception. Thompson decided that a higher-level language was needed for the further development of UNIX. So, Thompson designed a small language named B based on BCPL, a systems programming language developed in the mid-1960s. BCPL, in turn, traces its ancestry to Algol 60, one of the earliest (and most influential) programming languages.

Ritchie soon joined the UNIX project and began programming in B. In 1970, Bell Labs acquired a PDP-11 for the UNIX project. Once B was up and running on the PDP-11, Thompson rewrote a portion of UNIX in B. By 1971, it became apparent that B was not well-suited to the PDP-11, so Ritchie began to develop an extended version of B. He called his language NB (“New B”) at first, and then, as it began to diverge more from B, he changed the name to C. The language was stable enough by 1973 that UNIX could be rewritten in C. The switch to C provided an important benefit: portability. By writing C compilers for other computers at Bell Labs, the team could get UNIX running on those machines as well.

1.1.2. Chronological evolution of C language



1.1.3. History of C : Standardization (page 1)

C continued to evolve during the 1970s, especially between 1977 and 1979. It was during this period that the first book on C appeared. The C Programming Language, written by Brian Kernighan and Dennis Ritchie and published in 1978, quickly became the bible of C programmers. In the absence of an official standard for C, this book—known as K&R or the “White Book” to aficionados—served as a de facto standard.

During the 1970s, there were relatively few C programmers, and most of them were UNIX users. By the 1980s, however, C had expanded beyond the narrow confines of the UNIX world. C compilers became available on a variety of machines running under different operating systems. In particular, C began to establish itself on the fast-growing IBM PC platform.

With C’s increasing popularity came problems. Programmers who wrote new C compilers relied on K&R as a reference. Unfortunately, K&R was fuzzy about some language features, so compilers often treated these features differently. Also, K&R failed to make a clear distinction between which features belonged to C and which were part of UNIX. To make matters worse, C continued to change after K&R was published, with new features being added and a few older features removed. The need for a thorough, precise, and up-to-date description of the language soon became apparent. Without such a standard, numerous dialects would have arisen, threatening the portability of C programs, one of the language’s major strengths.

1.1.3. History of C : Standardization (page 2)

The development of a U.S. standard for C began in 1983 under the auspices of the American National Standards Institute (ANSI). After many revisions, the standard was completed in 1988 and formally approved in December 1989 as ANSI standard X3.159-1989. In 1990, it was approved by the International Organization for Standardization (ISO) as international standard ISO/IEC 9899:1990. This version of the language is usually referred to as C89 or C90, to distinguish it from the original version of C, often called K&R C. Appendix C summarizes the major differences between C89 and K&R C.

The language underwent a few changes in 1995 (described in a document known as Amendment 1). More significant changes occurred with the publication of a new standard, ISO/IEC 9899:1999, in 1999.
The language described in this standard is commonly known as C99. The terms “ANSI C,” “ANSI/ISO C,” and “ISO C”—once used to describe C89—are now ambiguous, thanks to the existence of two standards.

Because C99 isn’t yet universal, and because of the need to maintain millions (if not billions) of lines of code written in older versions of C, I’ll use a special icon (shown in the left margin) to mark discussions of features that were added in C99. A compiler that doesn’t recognize these features isn’t “C99-compliant.” If history is any guide, it will be some years before all C compilers are C99-compliant, if they ever are. Appendix B lists the major differences between C99 and C89.

CURRENT STANDARD ⇒ C11 ⇒ Yeah, you guessed it! (2011, C-Standard)

1.1.3. History of C : Standardization (page 3) `ivan's lines`

List of Standards

1. *K&R “de facto White Book” —---> 1978*
2. *C89/C90/K&R C —----> ANSI* standard approved in 1989*
3. *ANSI C/C99 —---> New standard in 1999 by ANSI*
4. *Current Standard is : C11*

*ANSI (American National Standards Institute)

1.1.4. History of C : C-Based Languages `ivan's lines`

- `C++` *includes all the features of C, but adds feature support for object oriented programming*
- `Java` *based on* `C++` *and therefore inherits many features*
- `C#` *is more recent language derived from* `Java` *and* `C++`
- `Perl` *another scripting language, adopted many features of* `C`
- `JavaScript` *uses syntax influenced by* `C`

1.2. Strength & Weaknesses of C `ivan's lines`

- Low level language
  - *Provides access to machine level concepts*
- Comparatively small
  - *C keeps itself limited by relying heavily on a library of standard functions*
- Permissive
  - *Provides more freedom compared to other higher level languages (which is also more risky)*

Chapter 2
C Fundamentals

CSE 4107 : Structured Programming I
REYNEP

Hello World!

```c
// hello.c

#include <stdio.h>

int main(void) {
    printf("hello, world\n");
}
```



2.1.1. Writing a Simple Program : Compiling and Linking (page 1)

The code, from last page, we need to write it down into a file, say, we name it `hello.c`. Despite its brevity, getting `hello.c` to run is more involved than you might expect.

First, we need to create a file named `hello.c` containing the program (any text editor will do). The name of the file doesn't matter, you can name it anything, but the `.c` extension is often required by compilers.

Next, we've got to convert the program to a form that the machine can execute. For a C program, that usually involves 3 steps:

1. `PreProcessing`
   - *The program is first given to a* preprocessor
   - preprocessor *obeys stuff that begin with* `#` *e.g.*
     - `#include` *is called a* "directive" / "command"
     - `#include <stdio.h>`
   - *A* preprocessor *is a bit like* "an editor"*;*
     - *it can add/*copy-paste*/concatenate things to the* `.c` *file and thus making* modifications*.*
     - `#include <stdio.h>`
       - copy-pastes *all the content of a file called* "stdio.h"
2. `Compiling`
   - *The preprocessor-*modified `.c` *file now goes to a* compiler*.*
   - Compiler *translates our* `hello.c` *into* "binary machine instructions"/ `machine-code` / `object-code`.
     - *e.g.* `hello.c` ➡️ `hello.obj`
   - *However, The program isn't quite ready to run just yeet!*
3. `Linking`
   - *This is the final step & is required, if we wanna run our* `hello.c`.
   - *A* linker *combines the* `hello.obj` *with other* `.obj` *files e.g.*
     - *we used* `printf()`.
     - `printf()` *'s binary code is inside another* `.obj` *provided by our OS.*
     - Linker *links that binary code with* `hello.obj`
     - *thus yielding a complete executable program.*

2.1.2. Writing a Simple Program : Compiling and Linking (page 2)

Fortunately, this process is often automated, so you won't find it too onerous.
In fact, the preprocessor is usually integrated with the compiler, so you probably won't even notice it at work.

The commands necessary to compile and link vary, depending on the compiler and operating system. Under UNIX, the C compiler is usually named `cc`. To compile and link the `hello.c` program, enter the following command in a terminal or command-line window:

```
cc hello.c
    # PreProcessor is usually integrated with compiler
    # Linking is automatic when using cc; no separate link command is necessary.
```

After **compiling** and **linking** `hello.c`

- `cc` *leaves the* "compiled" `binary-code` *in a file named* `a.out` *by default.*
- `cc` *has many options, tho.*
  - *one of them* ➡️ `-o` *allows us to choose the name of the output file. e.g.*

    ```
    cc hello.c -o hello         # Linux
    cc hello.c -o hello.exe     # Windows, (altho, windows wouldnt have `cc` by default)
    ```

2.1.3. Writing a Simple Program : Compiling and Linking (page 3)

The GCC Compiler

One of the most popular C compilers is the `GCC compiler`, which is supplied with **Linux** but is available for many other platforms as well. Using this compiler is similar to using the traditional **UNIX** `cc` compiler. For example, to compile the `hello.c` program, we would use the following command:

```
gcc hello.c -o hello         # Linux
gcc hello.c -o hello.exe     # Windows (available via, `msys2` + `mingw64`)
```

Others

1. `cc` *: not a compiler itself, rather just a* "Environment Variable"
2. `gcc` *: GNU C Compiler (Linux/Unix)*
3. `clang` *: Apple's C Compiler*
4. `msvc` *: Microsoft Visual C (only for Windows)*

2.1.4. Summary 🟪 `reynep's`

1. `PreProcessing`
   - `#include` = "directive" / "command" - *(starts with* `#` *)*
   - `#include <stdio.h>`
     - copy-pastes *all the content of a file called* "stdio.h"
2. `Compiling`
   - Compiler *translates* `hello.c` *into* "binary machine instructions"
     - *e.g.* `hello.c` ➡️ `hello.obj`
3. `Linking`
   - linker *combines the* `hello.obj` *with say* `stdio.obj`
4. `gcc`
   - `gcc` / `cc` / `clang` / `msvc` *does all 3 steps, then, after* linking, *hands us* `hello` / `hello.exe`

     ```
     gcc hello.c -o hello         # Linux
     gcc hello.c -o hello.exe     # Windows (available via, `msys2` + `mingw64`)
     ```

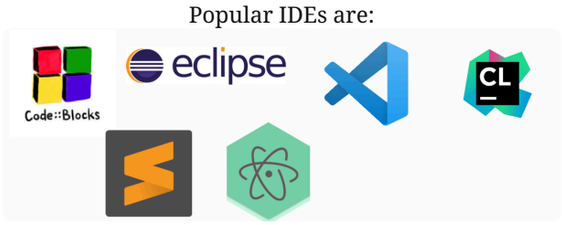
2.1.5. Writing a Simple Program : Integrated Development Environments

So far, we've assumed the use of `gcc` & `cc`. Those can only be used inside the `terminal` / `command-line`. But, we do have an better alternative.

- *The alternative is to use an integrated development environment (IDE)*
  - *IDE is A software package that allows us to*
    - edit
    - compile
    - link
    - execute/run &
    - highlight syntax with colors
    - *even* "debug" *a program*
  - *The components of an IDE are designed to work together. For example,*
    - *when the compiler detects an error in a program, it can arrange for the editor to highlight the line that contains the error.*

There's a great deal of variation among IDEs, so I won't discuss them further in this book. However, I would recommend checking to see which IDEs are available for your platform.

Popular IDEs are:



2.2. General Form

```c
// hello.c

#include <stdio.h>

int main(void) {
    printf("hello, world\n");
}
```

```c
// general form

#preprocessor directives

int main(void) {
    statements
}
```

> **REYNEP's Note on BackStory**
>
> K.N. King's TextBook was written in 1996. 2nd edition came out in 2008, and it's hard to change every single line of the book. So, I believe, the author kinda kept a few lines from 1996 here.
>
> Back then, towards the end of 1980s, in universities and high schools, for almost 70-80% of the courses, `Pascal` / `Turbo Pascal` was used! And the most popular / widely used programming languages were:
> - `FORTRAN` *(1957)*,
> - `COBOL` *(1960)*, `BASIC` *(1964)*
> - `Pascal` *(1970)*, `Turbo Pascal` *(1983)*
> - `Ada` *(1980)*, `SQL` *(1973)*
>
> Now, except for FORTRAN, All of these kind of used `begin` & `end` instead of `{}`.

Now here's the text that the author wrote back in 1996 (I presume).

> Notice how the `{}` braces show where `main()` **begins** and **ends**.
> Now, C uses `{` and `}` in much the same way that some other languages use words like `begin` and `end`.
>
> This illustrates a general point about C:
> - C *relies heavily on* abbreviations *and* special symbols
>   - *which is one reason that C programs are concise (or—less charitably—cryptic).*
> - *Even the simplest* C *programs rely on* 3 key features*:*
>   *i.* preprocessor-directives*: (editing commands that modify our* `hello.c` *program prior to compilation)*
>   - `#include <stdio.h>`
>   - `#include` *is called a* "directive" / "command"
>   - *"modify": as in* "copy-paste" *the entire content of a file called* "stdio.h"
>
>   *ii.* functions*: (named blocks of executable code)*
>   - `int main(void)` ➡️ *here* `main` *is a function*
>
>   *iii.* statements*: (commands to be performed when the program is run).*
>   - `printf("hello, world\n");` ➡️ *is a statement*
>
> We'll take a closer look at these features now.
>
> **REYNEP's Note**: Now, Shahriar Ivan kept these exact same lines, word by word in his slides. So i didn't modify/edit them.

2.2.1 General Form : `PreProcessor Directives`

Recall Back to 2.1.4. Summary `reynep's`

1. PreProcessing
   - `#include` = "directive" / "command" - *(starts with `#`)*
   - `#include <stdio.h>`
     - copy-pastes *all the content of a file called* "stdio.h"
2. Compiling
   - Compiler *translates* `hello.c` *into* "binary machine instructions"
     - *e.g.* `hello.c` ➡️ `hello.obj`
3. Linking
   - linker *combines the* `hello.obj` *with say* `stdio.obj`
4. gcc
   - `gcc` / `cc` / `clang` / `msvc` *does all 3 steps, then, after* linking, *hands us* `hello` / `hello.exe`

```
gcc hello.c -o hello         # Linux
gcc hello.c -o hello.exe     # Windows (available via, `msys2` + `mingw64`)
```

New in this Section

- `#include <stdio.h>`
  - *Before a* C *program is* compiled, *it is first edited by a* preprocessor.
    - *"edit": copy-paste/add/include*
  - *Commands intended for the preprocessor are called directives.*
    - *Chapters 14 and 15 discuss directives in detail.*
    - *For now, we're interested only in the* `#include` *directive.*
    - *e.g.*
      - `#include <stdio.h>`
  - `#include <stdio.h>`
    - copy-pastes *all the content of a file called* "stdio.h"
  - `<stdio.h>` *is called a* "header file"
    - *C has a number of* "headers"

```
Header
stdio.h    ➡️ printf(), scanf(), getchar() + more Input/Output stuffs.
stdbool.h  ➡️ bool, true, false
math.h     ➡️ sqrt(), pow(), sin(), cos()
ctype.h    ➡️ isdigit(), isalpha(), toupper(), tolower()
time.h     ➡️ time()
stdlib.h   ➡️ rand(), exit(), malloc(), free()
string.h   ➡️ strlen(), strcpy(), strcat()
... (more header-files)
```

      - *All of these header-files together is called* "the standard library (STL)".
      - `<stdio.h>` *contains information about* C's *standard I/O library.*
  - Preprocessor-Directives *always begin with a* `#` *character.*
    - *By default, directives are 1 line long*
    - *There's no semicolon or other special marker at the end of a directive.*

2.2.2 General Form : `Functions`

Function is a named block of executable code. **Functions** are like "procedures" or "subroutines" in other programming languages (as of 1996)—they're the building blocks from which programs are constructed. In fact, a `C` program is little more than a collection of functions. **Functions** fall into 2 categories:

1. *Those written by the programmer / User defined Functions*
2. *Those provided as part of the* `C` *implementation* ➡️ "library functions", *since they belong to a* "library" *of functions that are supplied with the* compiler.

- Functions
  - *Introduced in Chapter 09*
  - `int main(void)` *is a function*
    - *name of this specific function :* `main`
  - *The term "function" comes from mathematics:*
    - $f(x) = x + 1$
  - *A function that computes a value uses the return statement to specify what value it* "returns."
    - *(very dumb line to say right here)*
    - *we must check out Chapter 09, to actually understand* "return"

2.2.3 General Form : The `main` function

- *The* `main` *function is Mandatory*
  - *The name* `main` *is critical.*
    - *It can't be anything else like* `whatever`
    - *It can't even be* `MAIN` *or* `mAiN`
    - *It must be alphabetically* `main`
  - *Because, the people who made the* compiler *decided, that,* `main()` *is where the program should start. So like, when we run* `hello`*, Statements from inside* `main()` *starts to be executed one by one.*
- `return 0;`
  - *This won't be fully clear before* Chapter 9
  - *(but, Ivan kept 2 lines here.)*
  - `return 0;` *means, The Status Code "0" is handed back to the Terminal/Command-Line @ the end of the execution. 1980s convention was that people used to* `return 404;` *or say* `return -1;` *to mean certain kinds of errors.* `return 0` *means, everything is okay, normal program execution.*
  - *Yes,* `return 0;` *inside* `main()` *is not hardcode required.*
    - *But, many* compilers *will produce warning messages if no* `return` *statement is provided at the end of* `main()` function.

2.2.4 General Form : `Statements`

A statement is a **"command"** to be executed **"when the program runs"**. (unlike **preprocessor-directives/commands**). We'll explore statements later in the book, primarily in **Chapters 5 and 6**.

The `hello.c` program uses only two kinds of statements:

1. *the* `return` *statement*
2. *the* function *call.*

```
printf("Hello, World\n");
```

- *Asking a function to perform its assigned task is known as* "calling the function".

`C` requires that each statement end with a **semicolon**.

- *exception:* compound statement
  - *e.g.*
- *The* semicolon *shows the compiler where the statement ends;*

2.2.5 General Form : `Printing Strings`

`printf` is a powerful **function** that we'll examine in **Chapter 3**.
So far, we've only used `printf` to display a **"string literal"**—a series of characters enclosed in double quotation marks. When `printf` displays a string literal, it doesn't show the quotation marks.

`printf` doesn't automatically advance to the next output line when it finishes printing.

```
printf("Hello ");
printf("World\n");
    // output:
    //     Hello World      (1 line)
```

To instruct `printf` to advance one line, we must include `\n` (the new-line character) in the string (i.e. inside Quotation marks). Writing a new-line character terminates the current output line; subsequent output goes onto the next line.

2.3 Comments

- *Can appear in anywhere in the program*
- *Comments may extend over more than one line*

- **MultiLine Comment (C89 Style)**
  - *begins with* `/*` *and ends with* `*/`
  - *Multiline comments stands out better than* Single-Line *(see below) Comment*

```c
/*
    Name: hello.c
    Purpose: Prints Hello, world
    Author: 🤷
*/

#include <stdio.h>
int main(void)
{                               /* beginning */
    printf("Hello, world\n");   /* print function */
    return 0;
}
```

  - *Failure to end a comment can result in program* statements *being* commented out *causing your program to be incorrect.*

```c
/*
    Name: hello.c
    Purpose: Prints Hello, world
    Author: 🤷

#include <stdio.h>
int main(void)
{                               /* beginning */
    printf("Hello, world\n");   /* print function */
    return 0;
}
```

- **Single Line Comment (C99 Style)**
  - *begins with* `//`

```c
//  Name: hello.c
//  Purpose: Prints Hello, world
//  Author: 🤷

#include <stdio.h>
int main(void)
{                               // beginning
    printf("Hello, world\n");   // print function
    return 0;
}
```

  - *we don't specify smth, to mean end of comment.*
    Simply, everything to the right-side of `//` *will be* "comment".
    - *Thus,* Less error prone *: no chance of unterminated comment to consume part of a program*
    - *Simple and helpful because it ends automatically at the end of the line*

2.4. Variables and Assignment

2.4.1. `Types`

Few programs are as simple as the `hello.c` . Most programs need to perform a series of calculations before producing output, and thus need a way to store data temporarily during program execution. In `C` , as in most programming languages, these storage locations are called variables.

```c
int x = 10;
float y = 5.5;
```

Every variable must have a `type` , which specifies what kind of data it will hold. `C` has a wide variety of `types` . *For now, we'll limit ourselves to just two:* `int` *and* `float` .

- `int`
  - *Short for* "integer"
  - *can store a whole numbers such as 0, 1, 392, or –2553*
  - *Limited Range*
    - *Largest:* `2,147,483,647`
- `float`
  - *Short for* "floating point numbers"
  - *can store (theoretically) much larger decimal numbers than* `int`
  - *can store numbers with digits after the decimal point, like 379.125*
  - *Limitations:*
    - *Slower Arithmetic Operations (compared to* `int` *)*
    - *Round-off error*
      - *e.g. you can't store* `0.1000000159` *or stuff like that*
      - *instead* `0.100000016391277313232421875` *is stored*
      - *So,* `float` *has a nature of like, "Approximation"*

2.4.2. `printf`

- `printf` `ivan's lines`
  - `printf` *can be used to display the current value of a variable*

```c
int h = 70;                    // say 70inches = 5feet10inches
printf("Height: %d\n", h);
    // outputs:
    //      Height: 70
```

  - `%d` *is a placeholder indicating where the value of height is to be filled in*
  - `%d` *works for* `int`
  - `%f` *is required for* `float`
    - *By default,* `%f` *displays* six digits after the decimal points

```c
int profit = 2150.480000111f;
printf("Profit:   $ %f", profit);
printf("\n");
    // outputs:
    //      Profit:   $ 2150.480000
```

```c
int profit = 2150.480000111f;
printf("Profit:   $ %.2f", profit);
printf("\n");
    // outputs:
    //      Profit:   $ 2150.48
```

    - *To force* `%f` *to display* `k` *digits after the decimal point, put* `.k` between `%` *and* `f`

  - *multiple variables inside* single call *of* `printf` *:*

```c
int height = 8;                     int length = 12; int width = 10;
printf(“Height: %d Length: %d\n”, height, length);
```

    - *There's no limit to the number of variables that can be printed by a single call of printf:*

2.4.3. `Declarations`

```c
int main(void)
{
    declarations
    statements
}
```

```c
int main(void)
{
    int height = 8;
    int length = 12;
    int width = 10;
    float profit;

    int volume = height * length * width;
    printf("Volume: %d\n", volume);
    return 0;
}
```

- `Variables` 🟩
  - *must be declared before they are used*
  - *can be declared one at a time*

    ```c
    int height;
    float profit;
    ```

  - *several can be declared in the same line*

    ```c
    int height, length, width, volume;
    float profit, loss;
    ```

- `Assignment` 🟩
  - *is the process of assigning a value to a variable*

    ```c
    height = 8;
    profit = 100.0;
    ```

  - *declare first, then assign*

    ```c
    // Correct way
    int height;
        height = 8;
    ```

    ```c
    // Wrong way
    height = 8;
    int height;
    ```

  - *multiple variable declaration in 1 line:* caveat

    ```c
    // Prefferred way
    int length = 12, width = 10;
    ```

    ```c
    // Not wrong, but be careful
    int length, width = 10;
    ```

  - *floating point number assignments*
    - *It's best to append the letter f to a floating point constant*

    ```c
    float profit = 100.0f;
    ```

  - *Assigning* `int` *inside* `float` */ or /* *the opposite*

    ```c
    float y = 5;        // REYNEP: meh, it's okay 🤷
    int   x = 5.7;      // REYNEP: 5 will be stored inside 'x'. Decimal part is thrown away.
    ```

    - *ivan: Assigning* `int` *to float, or vice versa is possible but not recommended*

- `Operations` & More `Assignments` 🟩
  - *Once* assigned, *a* variable *can be used to* compute *the value of* another variable:

```
int height = 10, length = 20, width = 5, volume;
volume = height * length * width;                  // volume is now 1000
```

  - *The right side of an assignment can be a* "formula"
    - *involving* constants, variables, *and* operators (`+` `-` `*` `/` `%`)
    - "formula"
      - *is called* "expression" *in* `C` *terminology*

- `Uninitialized Variables`
  - *Some variables are automatically set to 0, when a program begins to execute, but most are not.*
  - *A variable that doesn't have a default value and hasn't yet been assigned a value by the program is said to be uninitialized.*

```
int height;
    // declared   but   uninitialized variable
int width = 50;
    // declared   and     initialized variable
```

  - *Attempting to access the value of an* uninitialized variable
    - *e.g. by displaying the variable using printf*
    - *using it in an expression*
  - *may yield an unpredictable result such as 2568, –30891, or some equally strange number.*
    With some compilers, worse behavior—even a program crash—may occur.

```
int height;
printf("%d", height);
    // might print 2568, might print -30891, might print 0.... so on.... no pattern
```

- `Operations` : `Caveat #1`

```
float x = 5/9;          // stores 1
float x = 5.0/9.0;      // stores 0.555555
float x = 5.0f/9.0f;    // stores 0.555555
```

2.4.4. Problem: `Computing the Dimensional Weight of a Box`

Shipping companies don't especially like boxes that are large but very light, since they take up valuable space in a truck or airplane. In fact, companies often charge extra for such a box, basing the fee on its volume instead of its weight. In the United States, the usual method is to divide the volume by 166 (the allowable number of cubic inches per pound). If this number—the box's "dimensional" or "volumetric" weight—exceeds its actual weight, the shipping fee is based on the dimensional weight. (The 166 divisor is for international shipments; the dimenional weight of a domestic shipment is typically calculated using 194 instead.)

Let's say that you've been hired by a shipping company to write a program that computes the dimensional weight of a box. Since you're new to `C`, you decide to start off by writing a program that calculates the dimensional weight of a particular box that's `12" × 10" × 8"`. Division is represented by `/` in `C`, so the obvious way to compute the dimensional weight would be `weight = volume / 166;`

where weight and volume are integer variables representing the box's weight and volume. Unfortunately, this formula isn't quite what we need. In `C`, when one integer is divided by another, the answer is "truncated": all digits after the decimal point are lost. The volume of a `12" × 10" × 8"` box will be 960 cubic inches. Dividing by 166 gives the answer 5 instead of 5.783, so we have in effect rounded down to the next lowest pound; the shipping company expects us to round up. One solution is to add 165 to the volume before dividing by 166: `weight = (volume + 165) / 166;`

A `volume of 166` would give a weight of `331/166`, or `1`, while a `volume of 167` would yield `332/166`, or `2`. Calculating the weight in this fashion gives us the following program.

```c
/* dweight.c */
/* Computes the dimensional weight of a 12" x 10" x 8" box */
#include <stdio.h>

int main(void)
{
    int height, length, width, volume, weight;

    height = 8;
    length = 12;
    width = 10;
    volume = height * length * width;
    weight = (volume + 165) / 166;
        // why? Read problem statement
        // here's a TL;DR tho
        // volume = 166   ==>   weight = 331/166 = 1
        // volume = 167   ==>   weight = 332/166 = 2

    printf("Dimensions: %dx%dx%d\n", length, width, height);
    printf("Volume (cubic inches): %d\n", volume);
    printf("Dimensional weight (pounds): %d\n", weight);

    return 0;
}
```

The output of the program is

```
Dimensions: 12x10x8
Volume (cubic inches): 960
Dimensional weight (pounds): 6
```

2.5. Reading Input

```c
#include <stdio.h>

int main(void)
{
    int x = 0;
    float y = 0;
    scanf("%d", &x);
    scanf("%f", &y);

    printf("Your Integer: %d", x);
    printf("Your   Float: %f", y);
}
```

- `scanf` *is the* `C` *library's counterpart to* `printf()`
- `scanf` *requires a* "format string" *to specify the appearance of the input data*
  - *a.k.a* `%d`
- *the* `&` *symbol is required when using* `scanf()`
- *We can modify our code from last page to take inputs from user using* `scanf()`

```c
    /* dweight.c */
    int height, length, width, volume, weight;

    printf("Enter height of the box: ");
    scanf("%d", &height);
    printf("Enter length of the box: ");
    scanf("%d", &length);
    printf("Enter width of the box: ");
    scanf("%d", &width);

    volume = height * length * width;
    weight = (volume + 165) / 166;
```

  - *output*

```
Enter height of the box: 8
Enter length of the box: 12
Enter width of the box: 10
Volume (cubic inches): 960
Dimensional weight (pounds): 6
```

2.6. `Defining` `Macros`

```c
#define RECIPROCAL_OF_PI (1.0f / 3.14159f)

#define INCHES_PER_POUND 166
#define ROUND_UP_OFFSET INCHES_PER_POUND-1

weight = (volume + ROUND_UP_OFFSET) / INCHES_PER_POUND;
```

- `#define` *is a* preprocessing directive,
  - *just as* `#include` *was*
  - *so there's no semicolon at the end of the line.*
  - *When a program is* compiled, *the* preprocessor *replaces each* "macro" *by the value that it represents.* e.g.

    ```c
    weight = (volume + INCHES_PER_POUND - 1) / INCHES_PER_POUND;
        // becomes
    weight = (volume + 166 - 1) / 166
        // after the preprocessing round
    ```

  - *The value of a macro can be an expression:*

    ```c
    #define RECIPROCAL_OF_PI (1.0f / 3.14159f)
    ```

    - *If it contains operators, the expression should be enclosed in parentheses.*

When a program contains constants, it's often a good idea to give them names. The `dweight.c` and `dweight2.c` programs rely on the `constant 166`, whose meaning may not be at all clear to someone reading the program later. Using a feature known as **"macro definition"**, we can name this constant: `#define INCHES_PER_POUND 166`

Notice that we've used only upper-case letters in macro names. This is a convention that most `C` programmers follow, not a requirement of the language. (Still, `C` programmers have been doing this for decades; you wouldn't want to be the first to deviate.)

2.6.1. Problem: `Converting from Fahrenheit to Celsius`

The following program prompts the user to enter a Fahrenheit temperature; it then prints the equivalent Celsius temperature. The output of the program will have the following appearance (as usual, input entered by the user is underlined):

```
Enter Fahrenheit temperature: 212
Celsius equivalent: 100.0
```

The program will allow temperatures that aren't integers; that's why the Celsius temperature is displayed as 100.0 instead of 100. Let's look first at the entire program, then see how it's put together.

```c
/** celcius.c */
/* Converts a Fahrenheit temperature to Celsius */
#include <stdio.h>

#define FREEZING_PT 32.0f
#define SCALE_FACTOR (5.0f / 9.0f)

int main(void)
{
    float fahrenheit, celsius;
    printf("Enter Fahrenheit temperature: ");
    scanf("%f", &fahrenheit);

    celsius = (fahrenheit - FREEZING_PT) * SCALE_FACTOR;

    printf("Celsius equivalent: %.1f\n", celsius);
    return 0;
}
```

The statement `celsius = (fahrenheit - FREEZING_PT) * SCALE_FACTOR;` converts the Fahrenheit temperature to Celsius. Since `FREEZING_PT` stands for `32.0f` and `SCALE_FACTOR` stands for `(5.0f / 9.0f)`, the compiler sees this statement as:

```c
celsius = (fahrenheit - FREEZING_PT) * SCALE_FACTOR;    // Before PreProcessing
celsius = (fahrenheit - 32.0f)       * (5.0f / 9.0f);         // After PreProcessing
```

Caveats

```c
#define SCALE_FACTOR (5.0f / 9.0f)
```

- *Defining* `SCALE_FACTOR` = `(5.0f / 9.0f)` *instead of* `(5 / 9)` *is important, because* `C` "truncates" `5/9` *into just* `1` *a.k.a* "truncated" *when two integers are divided.* "truncation" *is kinda like* mathematical "floor"ing

```c
printf("Celsius equivalent: %.1f\n", celsius);
```

- *Notice the use of* `%.1f` *to display celsius with just one digit after the decimal point.*

2.7. Identifiers 🟩 `ivan's lines`

- *Names of* variables, functions, macros *(+ a few more other entities) are called* "identifiers"

- 🎄 *An* "identifier" *may contain* `letters`, `digits`, *and* `underscores`.
- 🎄 *An* "identifier" *must begin with a* `letter` / `underscore`.
  - `times10`
  - `get_next_char`
  - `_done`

- 🟥 *An* "identifier" cannot *begin with a digit.*
  - `10times`
  - `get-next-char`
    - *these are* illegal "identifiers"

- *C places no limit on the maximum length of an identifier*

- `C` *is* `case-sensitive`
  - `C` *distinguishes between upper-case and lower-case letters in* identifiers
  - *e.g.*
    - `job` != `joB` != `jOb` != `jOB` != `Job` != `JoB` != `JOb` != `JOB`
    - *all of them identifiers are all different*

  - `SnakeCase`
    - *Many programmers use only lower-case letters in* identifiers, *with underscores inserted for legibility*
    - `symbol_table`
    - `current_page`
    - `name_and_address`
    - *(other than macros)*

  - `CamelCase`
    - *Other programmers use an upper-case letter to begin each word within an identifier:*
    - `symbolTable`
    - `currentPage`
    - `nameAndAddress`

- *The following* `keywords` *can't be used as* identifiers:

```
auto      enum      restrict*  unsigned
break     extern    return     void
case      float     short      volatile
char      for       signed     while
const     goto      sizeof     _Bool*
continue  if        static     _Complex*
default   inline*   struct     _Imaginary*
do        int       switch
double    long      typedef
else      register  union
```

```
EntryNotFound (FileSystemError): Error: ENOENT: no such file or directory, open 'c:\Users\REY\Desktop\
```

Chapter 3
Formatted Input/Output

CSE 4107 : Structured Programming I
REYNEP

3.0. characters

@credit *cs50*

```c
char c = 'A'
char c = 65;  // same effect as last line

char l = 'b';
char l = 98;  // same effect as last line
```

```c
for (int i = 0; i < n; i++)
{
    char c = str[0];     // if c = '5'
     int k = '0' - c;    //    k = 5;
}
```

ASCII

characters

- `char` ➡️ *8-bit*

3.1. The `printf()` function

```c
int i = 10;
int j = 20;
float x = 43.2892f;
float y = 5527.0f;

printf("hello, world!\n");
printf("i = %d, j = %i, x = %f, y = %f\n", i, j, x, y);
```

```
// output
hello, world!
i = 10, j = 20, x = 43.289200, y = 5527.000000
```

"string"

`"hello, world!\n"`

- *This is called a* String
- *Whatever is* `"` *inside* `"` *Double-Quotes* ➡️ String
- `\n` - newline
  - *This is the* ASCII *Character* `10`
  - *Without this, there'd be no 2nd line*

"conversion specification"

- `%d` *- decimal*
- `%i` *- integer*
- `%f` *- float*
  - *These are called* conversion specification
  - *They start with a* `%`
  - *They must stay inside* String
  - *They get "filled in" with values of* `i`, `j`, `x`, `y` *during printing* 🤷‍♀️

"format string"

- *The* `printf()` *function takes a* string *as input*
- *now, because* `printf()` *has built-in functionality to handle* conversion specification *(* `%d`, `%` *stuffs)*
  - *this* string *gets a special name*
  - *this* string *is called* "format string"

- *3 kinds of stuff that can be inside this* "format string"
  i. "ordinary characters" ➡️ *letters/buttons of a keyboard*
    - *simply, English Letters, Digits*
    - *stuff from the ASCII table*
  ii. "escape sequence" ➡️ *stuff like* `\n`
    - `\n` *is also another ASCII character*
  iii. "conversion specifications" ➡️ `%d`, `%` *stuffs*
    - *by using* `%d`, `%` *stuffs, we can insert values @ specified points in the* string *& those values will be printed*

using `%d`

```c
int x = 50;

printf("welcome to cs%d", x);
printf("\n");
```

1.

```c
int x = 10;
int y = 20;
printf("x = %d, y = %d", x, y);
printf("\n");
```

2.

1. *we used 1* `%d` *, so we passed in 1 integer right after the* string
2. *we used 2* `%d` *, so we passed in 2 integer right after the* string
   - *so, like, formally,*
     - *the values of* `x` *,* `y` *are gonna be "inserted into" the* format string*, "left-to-right" @ the places of* conversion specification

- *There's no limit on "the number of values" that can be printed by 1 single call of* `printf()`

ivan's lines

- `printf()` *must be supplied with the* format string*,*
  followed by any values that are to be inserted into the string *during printing:*

```c
printf(string, expr1, expr2, …);
```

- *The* format string *may contain both*
  - "ordinary characters" ➡️ *letters/buttons of a keyboard*
  - "conversion specification" ➡️ *which begin with the* `%` *character.*
- *A* conversion specification *is a placeholder representing a value to be filled in during printing*
  - `%d` *is used for* `int` *values*
  - `%i` *is used for* `int` *values*
  - `%f` *is used for* `float` *values*
- *Then he shows the code @ the start of this page, & typed beside*
  - "ordinary characters" *in a* format string *are printed as they appear in the* string*;*
  - "conversion specifications" *are replaced*

Possible Errors

1.

```c
printf("%d %d\n", i);          /*** Too many conversion specifications ***/
```

```
outputs:
   50 1561391600
   50 <some-random-int>
```

- Compilers *are not required to check if the number of* conversion specification *in a* format string *matches the number of output items.*

2.

```c
printf("%d\n", i, j);          /*** Too few  conversion specifications ***/
```

```
outputs:
   50
```

- *works,* `j` *is just ignored.*

3.

```c
int k = 50;
float y = 3.14;
printf("%f %d\n", k, y);       /***   wrong  conversion specifications ***/
```

```
outputs:
   0.000000 1610612736
```

- `float` *is stored way differently than* `int`
  - *so, when a really small* `float` *like* `3.14` *is "interpreted as"* `int` *, smth like* `1610612736` *comes out!*
    - *same goes for the other way around.*
- *If the programmer uses an incorrect* conversion specification*, the program will produce meaningless output, like in this case.*
- Compilers *are not required to check if a* conversion specification *is appropriate*

3.1.1. "Conversion Specifications" = "Format Specifiers"

- *ivan calls these* "Conversion Specifications"*, by* "Format Specifiers"
  - *iguess, because they specify stuff inside* "Format Strings"
  - *actually, the whole world calls these* `%` *stuffs* "Format Specifiers"

%ক

- Numbers

%ক	Bits	Description	Range / Notes	%llক	Bits	Range / Notes
%i	32-bit	integer	max: $2^{31} - 1$	%lli	64-bit	max: $2^{63} - 1$
%u	32-bit	Unsigned int	Positive or zero	%llu	64-bit	
%d	32-bit	Decimal int - Same as %i		%lld	64-bit	
%x	32-bit	Hexadecimal (Lower-case)		%llx	64-bit	
%X	32-bit	Hexadecimal (Upper-case)		%llX	64-bit	
%o	32-bit	Octal		%llo	64-bit	
%f	32-bit	float	max ≈ 3.4×10^{38}	%lf	64-bit	max ≈ 1.8×10^{308}
%a	32-bit	Hex float	C99 style	%la	64-bit	

  - *we &* `ivan` *will only use* `%i` , `%d` & `%f`
  - `64-bit`
    - *integer :* `%lli` , `%lld` , `%llu`
    - *float :* `%lf`
  - `16-bit`
    - `%hi` , `%hd` , `%hu`

- Scientific Notation

%ক	Bits	Description
%e	64-bit	Scientific (Lower-case)
%E	64-bit	Scientific (Upper-case)
%g	64-bit	Shortest of %f or %e

Char, Letters, Strings

%ক	Bits	Description	Range / Notes
%c	8-bit	char	0–127 (ASCII)
%s		char[] str	
%p	64-bit	pointer	

- Tricks - *that we won't be using / that might confuse us*

Specifier	Description
%#x	Hex with 0x prefix
%+d	Show sign for positive
%05d	Zero-padded width 5

  - `%n` - *Saves The Number of Characters Printed so far by current call of printf() into variable*

```c
int k = 40; float x = 839.21f;

printf("|%05d|\n", k);
printf("|%+d|\n", k);
```

```
// outputs

|00040|
|+40|
```

3.1.2. Gaps

```c
int k = 40;
printf("|%d|%5d|%-5d|%5.3d|-%5.3d|\n", k, k, k, k, k);
```

```
// outputs: using '·' to represent '<space>'
|40|···40|40···|··040|040··|
```

- `%d` - `int` *using minimum amount of space*
- `%5d` - `int` *using minimum of 5 characters.* "Right-Justified" *[1]*
- `%-5d` - `int` *using minimum of 5 characters.* "Left-Justified"
- `%5.3d` - `int` *using minimum of 5 characters.* "Right-Justified". *At Least 3 Digits. [2]*
- `%-5.3d` - `int` *using minimum of 5 characters.* "Left-Justified". *At Least 3 Digits.*

- *[1]*
  - *Since variable* `k` *requires only two characters* `40`, three spaces *were added*
- *[2]*
  - *This one is actually confusing.*
  - *So, do some testing if you need.*
  - *Adds extra* "Leading Zeroes" *if needed*

```c
float x = 839.21f;
printf("|%f|\n", x);
printf("|%10f|\n", x);
printf("|%10.3f|%10.3e|%-10g|%-12f\n", x, x, x, x);
```

```
// using '·' to represent '<space>'
|839.210022|
|839.210022|
|···839.210|·8.392e+02|839.21···|839.210022··|
```

- `%f` - `float` *using minimum amount of space, to show the* "actual value"
  - *for our case it shows* `839.210022`
  - *cz binary can't store* `839.21` *exactly. The closest it can store is* `839.210022`
  - *This approximation is called* "floating point error margin"

- `%10f` - `float` *using minimum of 10 characters.*
- `%-12f` - `float` *using minimum of 10 characters.* "Left-Justified"
- `%10.3f` - `float` *using minimum of 10 characters.* "Right-Justified". *At least 3 digits* "after `.`"
- `%10.3f` - `float` *6chars* `.` *3digits ---------- [6 + 3 + 1 for* `.` *= 10]*
- `%10.3e` - `float` *2chars* `.` *3digits* `e` ± `02`
- `%10.3e` - `float` *2chars* `.` *3digits* `e` *3chars*
- `%15.3e` - `float` *7chars* `.` *3digits* `e` *3chars*
- `%15.7e` - `float` *3chars* `.` *7digits* `e` *3chars*

- `%-10g` - `float` *chooses between* `%f` & `%e`. *Omits* "Trailing/Leading Zeroes"
  - *For our example case, chooses* `%f`
  - *Acts exactly like* `%-10.2f`

3.1.3. Gaps - Summaries

- `%5d` v/s `%10f`
  - `%5d` : `int` *using min. 5 chars.* "Right-Justified"
  - `%10f` : `float` *using min. 10 chars.* "Right-Justified". `.` *is an extra char which will always be there.*
  - `%10e` : `exponential` *using min. 10 chars.* "Right-Justified". `e+00` *are 4 extra char.*

- `%5.3d` v/s `%10.3f`
  - `%5.3d` : `int` *using min. 5 chars.* "Right-Justified". *At Least 3 Digits* "using `0`"
  - `%10.3f` : `float` *using min. 10 chars.* "Right-Justified". *At least 3 digits* "after `.`"

3.1.4. `___` $\%-n_1.n_2$क

ex.	generalized
`%i` , `%d` , `%u` , `%f`	%क
`%lli` , `%lld` , `%llf`	% *ll* क
`%5d` , `%15f`	% n_1क
`%5.3d` , `%10.3f`	% n_1 . n_2क
`%-5.3d` , `-%10.3f`	% − n_1 . n_2क

- *ll* : *64-bit stuffs*
- − : "Left-Justified"
- n_1 : *using minimum of* n_1 *chars. (inclusing* `.` *-decimal-point, or* `e+00` *)*
- n_2 : *at least* n_2 *digits*
  - `int` *at least* n_2 *digits* "using `0`"
  - `float` *at least* n_2 *digits* "after `.`"

```c
int k = 40;
float x = 839.21f;

printf("|%d|%5d|%-5d|%5.3d|-%5.3d|\n", k, k, k, k, k);
printf("|%f|\n", x);
printf("|%10f|\n", x);
printf("|%10.3f|%10.3e|%-10g|%-12f\n", x, x, x, x);
```

```
// outputs: using '·' to represent '<space>'


|40|···40|40···|··040|040··|
|839.210022|
|839.210022|
|···839.210|·8.392e+02|839.21···|839.210022··|
```

ivan's lines (edited) 🟩

- *A conversion specification can have the form* $\%n_1.n_2$क *or* $\%-n_1.n_2$क ,
  - क *is a letter.*
  - n_1 *and* n_2 *are integer* constants
  - n_1 *and* n_2 *are optional*
    - *if* n_2 *is omitted then, the* `.` *dot that separates* n_1 *and* n_2 *is also dropped*

- `%10.2f`
  - n_1 *is* `10`
  - n_2 *is* `2`
  - क *is* `f`
- `%10f`
  - n_1 *is* `10`
  - n_2 *is missing (so is the* `.` *)*
  - क *is* `f`
- `%.2f`
  - n_1 *is missing*
  - n_2 *is* `2`
  - क *is* `f`

- $\%n_1.n_2$क $\implies$ "format specifier" $\implies$ $\%m.pX$
  - $n_1 \to m$ $\implies$ "minimum field width"
  - $n_2 \to p$ $\implies$ "precision"
  - क $\to X$ $\implies$ "conversion specification"
    - *I hate ivan's decision of using 'X' capital-x here, because,* `%X` *actually has it's own actual meaning "*Hexadecimal (Upper-Case)"

ivan's lines : (unedited) - details

- *Slide 6*
  - *A* "conversion specification" *can have the form* `%m.pX` *or* `%-m.pX`, *where* `m` *and* `p are integer constants and` *X is a letter*
  - *Both* `m` *and* `p` *are optional; if* `p` *is omitted, the period that separates* `m` *and p is also dropped*
  - *In the conversion specification* `%10.2f`, `m` *is* `10`, `p` *is* `2`, *and* `X` *is* `f`
  - *In the specification* `%10f`, `m` *is* `10` *and* `p` *(along with the* `.` *period) is missing,*
    - *but in the specification* `%.2f`, `p` *is* `2` *and* `m` *is missing*

- *Slide 7+8 :* $n_1 \rightarrow m$
  - *The* "minimum field width", `m`, *specifies the minimum number of characters to print*
  - *If the value to be printed requires* fewer than m *characters, it is* "left/right-justified" *within the field.*
    - `%5d` - "Right-Justified"
    - `%-5d` - "Left-Justified"
    - `%4d` *displays the number* `123` *as* `•123`.
      - `•` *represents the space character*
  - *If the value to be printed requires* more than m *characters, the field width automatically expands to the necessary size*
    - *don't get confused*
    - *this simply means, in this case,*
      - `%5d` *for* `123456` *the effect is same as* `%d`
  - `%-5d` *Putting a minus sign in front of* m *causes* "left justification"
    - *The specification* `%-4d` *would display* `123` *as* `123•`

- *Slide 9 :* $n_2 \rightarrow p$
  - *The meaning of the* "precision", p, *depends on the choice of* X, *the* "conversion specifier"
  - `%.3d` - *The* `d` *specifier is used to display an integer in decimal form.*
    - p *indicates the* "minimum number of digits" *to display (extra zeros are added to the beginning of the number if necessary).*
    - *If* p *is omitted, it is assumed to be 1*

- *Slide 10 :* $n_2 \rightarrow p$
  - `%e` - *Exponential format*
    - p *indicates how many digits should appear after the decimal point (the default is* `6` *including* `e+00` *???). If* p *is 0, no decimal point is displayed.*
  - `%f` - *Float - Non-Exponential Format (ivan calls 'non-exponential' = 'fixed decimal')*
    - p *has the same meaning as above*
  - `%d` - *Integer*
    - *Minimum number (*p*) of digits to display (append extra zeros at the beginning if needed)*

- *Slide 11*
  - *Show the use of format specifiers for both* `int` *and* `float`
    - *Use* `%d`, `%5d`, `%-5d`, `%5.3d` *for* `int k = 40`
    - *Use* `%10.3f`, `%10.3e` *for* `float y = 839.21`

3.1.5. Escape Sequence

```
printf("Item\tUnit\tPurchase\n\tPrice\tDate\n");
```

```
Item    Unit    Purchase
        Price   Date
```

- *The* `\n` *code that we used in format strings since day 1, is called an* "escape sequence"
- *Control Characters:*
  - *Alert (Bell) :* `\a` *- printing* `\a` *causes an audible beep on most machines. (well, REYNEP never saw it. 🥺)*
  - *NewLine :* `\n` *- printing* `\n` *advances the cursor to the beginning of the next line.*
  - *Horizontal Tab :* `\t` *- printing* `\t` *moves the cursor to the next tab stop*
  - *BackSpace :* `\b` *- printing* `\b` *moves the cursor back one position*

- *Special Characters:*
  - *Backslash :* `\\` - `printf(" ---- \\ ---- ");`
  - *Double Qotes :* `\"` - `printf(" ---- \" ---- ");`

3.1.6. White-Space Characters

- ` ` - ASCII 32 - **space**
- `\n` - ASCII 10 - **newline** - *Moves cursor to the beginning of the next line*
- `\t` - ASCII 9 - **horizontal tab**
- `\v` - ASCII 11 - **vertical tab**
- `\f` - ASCII 12 - **form feed** - *Advances the output to the next page or form*
- `\r` - ASCII 13 - **carriage return** - *Moves cursor to the beginning of the current line*

3.2.0. Memory

In weeks past, you may recall our "artist rendering" of concurrent blocks of memory.

Applying hexadecimal numbering to each of these blocks of memory, you can visualize these as follows:

You can imagine how there may be confusion regarding:

- *whether the memory-block labeled* `10` *in the first picture above*
  - *may represent a location in memory, or*
  - *may represent the value 10.*

Accordingly, by convention, all hexadecimal numbers are often represented with the `0x` prefix as follows:

```c
// Prints an integer
#include <stdio.h>

int main(void)
{
    int n = 50;
    printf("%i\n", n);
}
```

Notice how `n` is stored in memory with the value 50.
You can visualize how this program stores this value as follows

---

Notice how `n` is stored in memory.

- `value(n) = 50`
- `address(n) = 0x123`
  - *There is a way we can Print this Address.*

```c
// Prints an integer's address
#include <stdio.h>

int main(void)
{
    int n = 50;
    printf("%p\n", &n);
}
```

3.2.1. Pointers

The C language has two powerful **operators** that relate to memory:

- `&` - *Provides the address of something stored in memory. [**this page**]*
- `*` - *Instructs the compiler to go to a location in memory. [**next page**]*

```c
// Prints an integer's address
#include <stdio.h>

int main(void)
{
    int n = 50;
    printf("%p\n", &n);
}
```

`n` is stored in memory.

- `value(n) = 50`
- `address(n) = 0x123`
  - *There is a way we can Print this Address.*
  - *won't stay* "invariant"*, if we run it again.*

Things to notice:

- `%p`*, which allows us to view the address of a location in memory.*
- `&n` *can be literally translated as "the address of n."*
- `printf()` *line prints the integer's address.*

Executing this code will print an address of memory beginning with `0x`

> Def ^{n}
>
> > A **"pointer"** is a variable that stores the address of something.

```c
int  n = 50;
int *p = &n;
```

`p` is a pointer that contains the address of an integer `n`
Here, `*` is not an "operator"

- *wait a lil' bit before we get to that* `*` *"operator"*

```c
// Prints an integer's address
#include <stdio.h>

int main(void)
{
    int n = 50;

    printf("%p\n", &n);
}
```

- *previous code*

```c
// Stores and prints an integer's address
#include <stdio.h>

int main(void)
{
    int  n = 50;
    int *p = &n;
    printf("%p\n", p);
}
```

- *modified new code ➡️ has the same effect*

We have simply leveraged our new knowledge of the `&` operator.

Notice that the **pointer** seems rather large

- *it looks double sized in the picture*
- *Indeed, a pointer is usually stored as an 8-byte value.*
  - *8-byte = 64-bit*
  - *Why? ➡️ 🔗LinusTechTips*
- *maybe, by chance* `p` *ends up, here, sort of higher up on my slide*

Modified New Code $\iff$ Memory

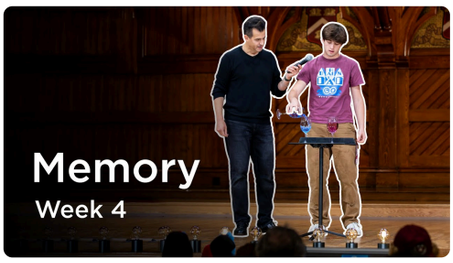
- `int *p` *creates a pointer*
  - *whose job is to store the memory address of an integer.*
- `int *p` *is a variable itself tho*
  - *var* `p` *is storing the address of var* `n`
  - *var* `p` *is storing the address of the* `50`

It is **Arcane**
It's a little weird

3.2.2. Pointers - `*` Operator - page1

So, What is `p`?
- *it's just a variable*
  - *whose value happens to be an address*

So, What is `n`?
- *it simply happens to be*
  - *an integer at that address*



@18m47s 🔗 - Pointers
@20m35s 🔗 - "Address Of" Operator `&` ➡️ `int *p = &n;`
@22m13s 🔗 - Printing a Pointer ➡️ `0x7ffef2c2925c`
@25m00s 🔗 - `int* p;` or `int *p;` ?
@26m00s 🔗 - "De-Reference" Operator `*`
@27m45s 🔗 - *The designers of C, decages ago, made an "unfortunate decision"....*
@29m00s 🔗 - in terms of memory....
@30m16s 🔗 - FAQ

@33m00s - little smth to not lose today's memory

I. REYNEP. Highly recommend that you watch a few moments from here....

3.2.2. Pointers - `*` Operator - page2

```c
// prints an integer
#include <stdio.h>

int main(void)
{
  int  n = 50;

  printf("%i\n", n);    // we can change this line ➡️
}
```

1. *how we normally print* `int n;`

```c
// prints an integer
#include <stdio.h>

int main(void)
{
  int  n = 50;
  int *p = &n;
  printf("%i\n", *p);
}
```

2. `*p` ⟹ *use the location of* `n` *called* `p` *... and "go there", i.e. follow the breadcrumbs*

Two Operators
1. "Address Of" *Operator* `&`
2. "De-Reference" *Operator* `*`

The designers of *C*, decages ago, made, I would say the "unfortunate decision", to use the `*` star in 3 different ways in total:
1. *multiplication*
2. "De-Referencing" ➡️ `printf("%i\n", *p);`
3. "Creating" *a pointer* ➡️ `int *p;`

Admittedly Confusing, but
- `int *p = &n;` ⟹ *we are declaring a* "pointer" *&* *assigning it a memory-address*
- `printf("%i\n", *p);` ⟹ *we are* "De-Referencing" *that pointer.*

- *one sort of "creates" the pointer*
- *one "goes to" that address*

Copying DAVID's Lines from *@26m00s* 🔗 - **"De-Reference" Operator** `*`

> Well,
>
> 1. *I could print out* `n` *literally as I'm doing now.*
> 2. *Or, I could use the location of* `n` *called* `p` *.... and "go there" .... with the .... 2nd of these two operators*
>   - *The* Dereference Operator.
>   - *The* Dereference Operator - `*` *means "go to this address"*
>   - *I'm kind of jumping through hoops proverbially here, because this is the stupid, longer, more complicated way to print out the value of 50, but it just speaks to the relative power that I have by just 1) figuring out what the address is and then 2) going there.*

Copying DAVID's Lines from *@27m45s* 🔗 - *The designers of C, decades ago, made an "unfortunate decision"....*

> But the confusing part I do think about this for me and for most folks learning this is the following:
>
> - *The designers of C, decades ago, made, I would say the "unfortunate decision", to use the* `*` *star in 3 different ways in total:*
>
> *i. multiplication*
>
> *ii.* `printf("%i\n", *p);` ➡️ *here it literally means "go there"*
>   - *i.e.* "DeReference" *the address*
>   - *i.e. follow the breadcrumbs, and go there.*
>
> *iii.* `int *p;`
>   - *But, when you declare the pointer*
>   - *when you create the pointer*
>   - *you also use a* `*` *but*
>   - *you also use the* `type` .
>
> - *So, what's confusing at least to me, is, we are using the* `*` *in two different ways*
>   - *both of which are related to addresses,*
>   - *both of which are related to pointers*
>
> 1. *But, when you create a variable or declare a variable for the first time,*
>   - `int *p;`
>   - *you specify not only* `*`
>   - *but the* `type` *of the pointer (just like any other variable declaration)*
> 2. *But, when you use the pointer subsequently,*
>   - `printf("%i\n", *p);`
>   - *you do not mention the data type again.*
>   - *The compiler is smart enough to "remember"*
>   - *"okay, got it. you wanted a pointer to an integer." from the line*
>   - `int *p;`

3.2.3. String

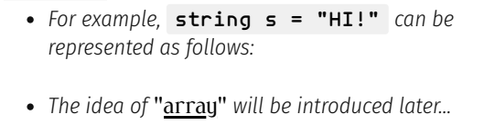
```c
// Prints a string
#include <cs50.h>
#include <stdio.h>

int main(void)
{
  string s = "HI!";  // comes from <cs50.h>
  printf("%s\n", s);
}
```

Now that we have a mental model for pointers, we can peel back a level of simplification that was offered earlier in this course.

Recall that a string is simply an "array" of characters.

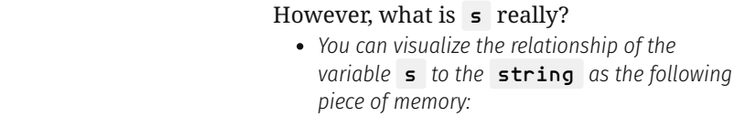
- *For example,* `string s = "HI!"` *can be represented as follows:*



- *The idea of* "array" *will be introduced later...*

However, what is `s` really?

- *You can visualize the relationship of the variable* `s` *to the* `string` *as the following piece of memory:*



Notice

- *how a pointer called* `s` *tells the compiler where the first byte of the string exists in memory*

The idea of "array" will be introduced later...

- *but the key idea is*
  - `s[0]` $:= char_0$ `= 'H'`
  - `s[1]` $:= char_1$ `= 'I'`
  - `s[2]` $:= char_2$ `= '!'` *& so on*
  - *in CS, we love to start from* `0`

```c
// Prints each character of a string
#include <cs50.h>
#include <stdio.h>

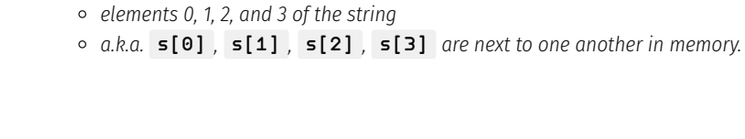
int main(void)
{
    string s = "HI!";
    char k0 = s[0];     printf("%c\n", k0);     // outputs: H
    char k1 = s[1];     printf("%c\n", k1);     // outputs: I
    char k2 = s[2];     printf("%c\n", k2);     // outputs: !
    char k3 = s[3];     printf("%c\n", k3);     // outputs:
}
```

```c
// Prints a string's address
    // as well the addresses of its chars
#include <cs50.h>
#include <stdio.h>

int main(void)
{
    string s = "HI!";
    printf("%p\n", s);
    printf("%p\n", &s[0]);
    printf("%p\n", &s[1]);
    printf("%p\n", &s[2]);
    printf("%p\n", &s[3]);
}
```

This code

- *prints the memory locations of each character in the* `string s`.
- *The* `&` *symbol is used to show the address of each element of the string.*



- *When running this code, & looking @ the output, notice that*
  - *elements 0, 1, 2, and 3 of the string*
  - *a.k.a.* `s[0]`, `s[1]`, `s[2]`, `s[3]` *are next to one another in memory.*

3.2.4. `char *str`

```c
// Declares a string with CS50 Library
#include <cs50.h>
#include <stdio.h>

int main(void)
{
    string s = "HI!";
    printf("%s\n", s);
}
```

1. *using the* $cs50$ *training wheel* `string`

```c
// Declares a string without CS50 Library

#include <stdio.h>

int main(void)
{
    char *s = "HI!";
    printf("%s\n", s);
}
```

2. `cs50.h` *is removed.* `string` = `char *`
   - `char *c` *will represent the* String *that starts at the location of* `s`.
   - `char *c`
     - *is raw* C *code,*
     - *is without the scaffolding of the* $cs50$ *library.*
     - *effectively removes the training-wheels of the* `string` *data type offered by* `cs50.h`.

3.2.5. String - Pointer Arithmetic

```c
// Prints each character of a string
#include <stdio.h>

int main(void)
{
    char *s = "HI!";
    char k0 = s[0];     printf("%c\n", k0);     // outputs: H
    char k1 = s[1];     printf("%c\n", k1);     // outputs: I
    char k2 = s[2];     printf("%c\n", k2);     // outputs: !
    char k3 = s[3];     printf("%c\n", k3);     // outputs:
}
```

```c
// Prints a string's chars
#include <stdio.h>

int main(void)
{
    char *s = "HI!";
    printf("%c\n", s[0]);   // same effect
    printf("%c\n", s[1]);
    printf("%c\n", s[2]);
    printf("%c\n", s[3]);
}
```

```c
// Prints a string's chars
#include <stdio.h>

int main(void)
{
    char *s = "HI!";
    printf("%c\n", s[0]);
    printf("%c\n", s[1]);
    printf("%c\n", s[2]);
}
```

1. *printing each* `char` @ `s`

```c
// Prints a string's chars via pointer arithmetic
#include <stdio.h>

int main(void)
{
    char *s = "HI!";
    printf("%c\n", *s);
    printf("%c\n", *(s + 1));       // Pointer Arithmetic
    printf("%c\n", *(s + 2));       // Eventually having the same effect
}
```

2. *Notice that*
   i. *the* 1st *character at the location of* `s` *is printed.*
   ii. *Then, the character at the location* `s + 1` *is printed, and so on.*
   iii. *effectly having the same effect as the code on the left*

3.2.6. Loops

```c
#include <stdio.h>

int main(void)
{
    printf("meow\n");
    printf("meow\n");
    printf("meow\n");
}
```



@1h23m26s - Loops

```c
#include <stdio.h>

int main(void)          // Code with Same Effect
{
    int i = 1;
    while (i <= 3)
    {
        printf("meow\n");
        i = i+1;
    }
}
```

- `i = i+1` ≔ *Every time* `1` *is added to* `i`
  - *Once* `i` *is greater than 3, it will stop the loop.*

```c
int i = 1;
while (i <= 3)
{
    printf("meow\n");
    i = i+1;
}
```

```c
int i = 0;
while (i < 3)
{
    printf("meow\n");
    i++;
}
```

3.2.6. Loops - Summary

1. repeats 3 times

```c
int n = 3;      // also we could have a variable like this

for (int i = 0; i < n; i++)     // that plug that in here
{
    printf("meow\n");
}
```

1. repeats forever

3.2.7. Functions

1. *These two left-images together*
   - *"no input"*
   - *"no output"*





```c
#include <stdio.h>

// Meow once
void meow(void)
{
    printf("meow\n");
}



int main(void)
{
    for (int i = 0; i < 3; i++)
    {
        meow();
    }
}
```

1. *These two right-images together*
   - `int n` *input*
   - *"no output"*





```c
#include <stdio.h>

// Meow some number of times
void meow(int n)
{
    for (int i = 0; i < n; i++)
    {
        printf("meow\n");
    }
}



int main(void)
{
    meow(3);
}
```

Printing

- *is* **not** *an "output" from the perspective of* "functions".
- *is an "output" from the perspective of user & the terminal*

"Functions"

- *inputs are called* "parameters" *-- can be multiple inputs*
- *output is called* "return value" *-- must return only 1 thing.*

- *Why do functions need inputs?*

```c
int main(void) {
    int k = 10;
}

int func(void) {
    printf("%d", k); printf("\n");
}
```

  - *Because, this code won't compile....*
  - `k` *is not accessible outside* `main`



@1h34m37s
Functions

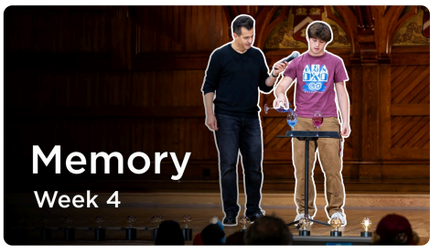
```c
// Meow some number of times
void meow(int n)
{
    for (int i = 0; i < n; i++)
    {
        printf("meow\n");
    }
}
```

```c
// Get number of meows
int get_positive_int(void)
{
    int n = -1;
    while (true)
    {
        n = get_int("Number: ");
        if (n > 1) {
            break;      // break the look
        }
    }
    return n;
}
```

```c
#include <cs50.h>
#include <stdio.h>

int main(void) {
    int n = get_positive_int();
    meow(n);
}
```

3.2.8. Swapping



In the real world

- *a common need in programming is to swap two values.*
- *Naturally, it's hard to swap two variables without a temporary holding space.*
- *In practice, you can type code* `swap.c` *and write code as follows to see this in action:*

@1h39m00s
swapping

The Problem

```c
// fails to swap two integers
#include <stdio.h>

void swap(int a, int b)
{
    int tmp = a;
    a = b;
    b = tmp;
}

int main(void)
{
    int x = 1;
    int y = 2;

    printf("x is %i, y is %i\n", x, y);
    swap(x, y);
    printf("x is %i, y is %i\n", x, y);
}
```

```
output
    x is 1, y is 2
    x is 1, y is 2
```

Notice that

- *while this code runs, it does not work.*
- *The values, even after being sent to the* swap() *function, do not swap.* Why?

When you pass values to a function,

- *you are only providing copies.*
- *In our code above,* `x` *and* `y` *are being passed by value.*
  - *whenever, this is done, values(* `x` & `y` *) are getting copied into* `a` & `b`

Two Different Functions

- `main` *and* `swap` *, in terms of memory,*
  - *have two separate frames or areas of memory.*

Image 1

- *executed the first 3 lines of* `main()`

Image 2

- *line 4 of* `main()` $\implies$ `swap(x, y);`
- `swap(x, y)` *has been called*
  - *so we must go inside this function*
  - *and execute code line by line*
  - *we executed line 1 inside* `swap()`

Image 3

- *line 2 inside* `swap` ➡️ *update* `a`
- *line 3 inside* `swap` ➡️ *update* `b`
- *no more code inside swap, the end*
  - *so we* `return`

Image 4

- *Even tho,* `a` & `b` *might have been swapped*
- `x` , `y` *was not changed* 🤷‍♀️

3.2.8. Swapping

The Solution

```c
// Swaps two integers using pointers
#include <stdio.h>

void swap(int *a, int *b)
{
    int tmp = *a;
    *a = *b;
    *b = tmp;
}

int main(void)
{
    int x = 1;
    int y = 2;

    printf("x is %i, y is %i\n", x, y);
    swap(&x, &y);
    printf("x is %i, y is %i\n", x, y);
}
```

Don't Get Confused!

Remember,
The designers of C, decages ago, made,
an "unfortunate decision":

- *Star* `*` *is used in 3 different ways in total:*
  i. **multiplication**
  ii. **De-Referencing** ➡ `printf("%i\n", *p);`
  iii. **Creating a pointer** ➡ `int *p;`

```c
void swap(int *a, int *b)
```

- `int *a` ➡ "Creating" *a pointer*

```c
int tmp = *a;
```

- `*a` ➡ "De-Referencing" *a pointer*

```c
*a = *b;
```

- `*a` ➡ *"follow the breadcrumbs", go to that address,* **save** *value in there*
- `*b` ➡ *"follow the breadcrumbs", go to that address,* **grab** *value from there*

This is called,

- *"passing value by reference"*
  - *That is*
    - *instead of copying values(* `x` & `y` *)*
    - *the addresses of* `x` *and* `y` *are provided to the function*
  - *And*
    - *if we know the address*
    - *we can just "go there" & poke around*

3.2.9. the `scanf()` function

```c
// Gets an int from user using scanf
#include <stdio.h>

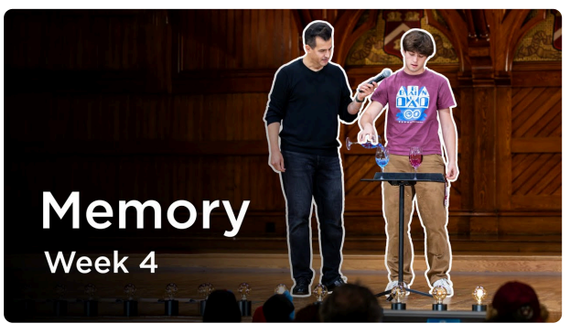
int main(void)
{
    printf("Enter a Number (n): ");
        int n;
        scanf("%i", &n);
            // %i ==> scanf() will expect an integer
            // &n ==> scanf() will store the integer @ the address "&n"
            //  n ==> is not a address, it's a variable
            // &n ==> is the actual address

    printf("n: %i\n", n);
}
```

Now, if you understood, why the swapping function needs address of `x` and `y`, then you could guess, why `scanf()` also needs address.

```c
int m, n;
float x, y;
scanf("%d %d %f %f", &m, &n, &x, &y);
```

```c
1 -20 .3 -4.0e3
  // Say, the user writes this line and hits enter
  // Then
        m = 1       x = 0.3
        n = -20     y = -4000.0
```



@2h01m24s
scanf()

```c
#include <stdio.h>

int main(void) {
    char *s;    // buggy
    printf("s: ");
    scanf("%s", s);
    printf("s: %s\n", s);
}
```

1. `s` *points to a different address everytime you run it*

```c
#include <stdio.h>

int main(void) {
    char s[4];  // fixed (somewhat)
    printf("s: ");
    scanf("%s", s);
    printf("s: %s\n", s);
}
```

2. `s` *is still a pointer.* `s` *now points to an* `char` *array of size* `4`
   - *we can input* `cat` *and the program functions.*
   - *However, an input larger than this could create an error.*
     - *so,* `char s[1024];` *or more could be done*

3.2.10. `scanf()` - ivans lines

Recall that
- `printf()` *prints output according to a really specific* **format string**
- `printf()` *prints output according to a specified format*

format string consists of:
- **ordinary characters**
- **conversion specifications**

Just the Same way
- `scanf()` *reads input according to a particular format too.*
  - *so* `scanf()` *takes in a* **format string** *too.*
  - *The conversions allowed with* `scanf()` *are essentially the same as those used with* `printf()`.
  - *A* `scanf()` **format string** *may contain both ordinary characters and conversion specifications.*

```
int m, n;
float x, y;
scanf("%d %d %f %f", &m, &n, &x, &y);
```

```
1 -20 .3 -4.0e3
```

- *(then up next,* **ivan** *showed this code & didn't even bother to write any text)*
- **ivan** *also called this input above, "output"*

In many cases
- *a* `scanf()` **format string** *will contain only these conversion specifications, as in the following example:*

Suppose that the user enters the following input line:
and then presses enter.
- `scanf()` *will read the line*
- *converting its characters to the numbers they represent,*
- *and then*
  - **assign** `1`, `-20`, `0.3`, *and* `-4000.0`
  - *to* `m`, `n`, `x`, *and* `y`, *respectively.*

```
scanf("%d %d %f %f", &m, &n, &x, &y);
scanf("%d%d%f%f", &m, &n, &x, &y);
```

Note that
- *both line here, would have had the same effect.*
  - *"Tightly packed"* **format strings** *like* `"%d%d%f%f"` *are common in* `scanf()` *calls.*

3.2.11. `scanf()` traps (unedited)

`scanf()` & `printf()` contains several traps for the unwary. When using them, the programmer must check that:

1. *the number of* **conversion specifications** *matches the number of* **input variables**
2. **each conversion** *is appropriate for the corresponding variable*
   - *the compiler isn't required to check for a possible mismatch.*
3. *Another trap for* `scanf()` *involves the* `&` *symbol*
   - `&` *normally precedes each variable in a* `scanf()` *call.*
   - *The* `&` *is usually (but not always) required, and it's the programmer's responsibility to remember to use it.*

3.2.12. How `scanf()` works 🟩 (unedfted)

Uhem,

- `scanf()` *can actually do much more than I've indicated so far.*
- `scanf()` *is essentially a* **"pattern-matching"** *function*
  - *that tries to match up groups of input characters with conversion specifications in a format-string*
- `scanf()` & `printf()` *is controlled by the format string.*

When `scanf()` is called,

- `scanf()` *begins processing the information in the string, starting at the left.*
- 🟩 *For each* **conversion specification** *in the format string*
  - `scanf()` *tries to locate an item of the appropriate type in the input data,*
    - *skipping blank space if necessary.*
  - `scanf()` *then reads the item*
    - *stopping when it encounters a character that can't possibly belong to the item.*
  - *If the item was read successfully,*
    - `scanf()` *continues processing the rest of the format string.*
  - *If any item is not read successfully,*
    - `scanf()` *returns immediately*
      - *without looking at the rest of the format string (or the remaining input data).*
  - *As* `scanf()` *searches for the beginning of a number, it* **ignores** white-space characters

---

- *Remember,* **white-space characters**
  - ` ` - ASCII 32 - **space**
  - `\n` - ASCII 10 - **newline** - *Moves cursor to the beginning of the next line*
  - `\t` - ASCII 9 - **horizontal tab**
  - `\v` - ASCII 11 - **vertical tab**
  - `\f` - ASCII 12 - **form feed** - *Advances the output to the next page or form*
  - `\r` - ASCII 13 - **carriage return** - *Moves cursor to the beginning of the current line*

- *As a result, numbers can be put on a single line or spread out over several lines.*
  Consider the following call of `scanf()` *:*

```
scanf("%d%d%f%f", &i, &j, &x, &y);
```

Suppose that the user enters three lines of input:

```
1
-20   .3
-4.0e3
```

`scanf()` *sees one continuous stream of characters:*

```
··1¤-20···.3¤···-4.0e3¤
```

- *Author's using* `·` *to represent space and* `¤` *to represent the new-line.*

```
··1¤-20···.3¤···-4.0e3¤
ssrsrrrssssrrssssrrrrrr
```

- `s` *under a character indicates that it was skipped*
- `r` *indicates it was read as part of an input item*

- 🎴 `scanf()` **"peeks"** *at the final* **new-line** *character without actually reading it.*
- 🎴 *This* **new-line** *will be the first character read by the next call of* `scanf()` *.*

3.2.13. How `scanf()` works - II - 🟩 (unedited)

What rules does `scanf()` follow to recognize an `int` or `float`?

- 🟩 *When asked to read an* `int`
  - `scanf()` *first searches for*
    - *a digit, or*
    - *a plus sign, or*
    - *a minus sign;*
  - *it then*
    - *keeps on reading digits until it reaches a non-digit.*
- 🟩 *When asked to read a* `float`
  - `scanf()` *first looks for*
    - *a digit, or*
    - *a plus sign, or*
    - *a minus sign;*
  - *followed by*
    - *more digits, or,*
    - *decimal point + more digits*
  - *followed by*
    - *an exponent (optional).*
    - *a.k.a.* `e+00` *or* `E+00`
      - *An exponent consists of the letter* `e` *(or* `E` *), an optional sign, and one or more digits.*
  - *The* `%e` *,* `%f` *, and* `%g` *conversions are interchangeable when used with* `scanf()` *;*
    - *all three follow the same rules for recognizing a* `float`

3.2.14. How `scanf()` works - III : Matching Conv. Specs. 🟦 TextBook

🎴 When `scanf()` encounters a character that can't be part of the current item,

- *the character is* **"put back"** *to be read again*
  - *during the scanning of the* **next input item** *(next* `%` *stuff) or*
  - *during the* **next call** *of* `scanf()` *.*

```
INPUT: 1-20.3-4.0e3¤

scanf("%d%d%f%f", &m, &n, &x, &y);
    // Let's use the same call of scanf as before
```

Consider the following (admittedly pathological) arrangement of our four numbers:

Here's how `scanf()` would process the new input:

1. `%d`
   - *The first nonblank input character is* `1` *;*
     - *since integers can begin with* `1` *, scanf then reads the next character,* `-` *.*
   - *Recognizing that* `-` *can't appear inside an integer,*
     - `scanf()` *stores* `1` *into* `m` *and*
     - *puts the* `-` *character back.*
       - *for it shall be processed by the next call of* `scanf()` *or the next* `%` *item.*
2. `%d`
   - `scanf()` *then reads the characters* `-` *,* `2` *,* `0` *, and* `.` *(period).*
     - *The 4th character it read was* `.` *Since an* `int` *can't contain a decimal point,*
       - *scanf stores* `-20` *into* `m` *and*
       - *puts the* `.` *character back.*
3. `%f`
   - `scanf()` *reads the characters* `.` *,* `3` *, and* `-` *.*
     - *Since a* `float` *can't contain a minus sign after a digit,*
       - `scanf()` *stores* `0.3` *into* `x` *and*
       - *puts the* `-` *character back.*
4. `%f` *- Last one,*
   - `scanf()` *reads the characters* `-` *,* `4` *,* `.` *,* `0` *,* `e` *,* `3` *, and* `¤` *(new-line).*
     - *Since a* `float` *can't contain a* **new-line** *character,*
       - `scanf()` *stores* -4.0×10^3 *into* `y` *and*
       - *puts the* **new-line** *character back.*
   - 🎴 *NOTE: this* `¤` *(new-line)* **will be read by the next** `scanf()`

In this example,

- `scanf()` *was able to match every* **conversion specification** *in the format string with an input item.*
- 🎴 *Since the* **new-line** *character wasn't read,*
  - **it will be left for the next call of** `scanf()` *.*

3.2.15. How `scanf()` works - IV : Ordinary Characters 🟩

Consider

```
scanf("%d SMTH %d", &x, &y);
scanf("%d/%d", &x, &y);
```

🎴 🟩 When `scanf()` encounters a character that can't be part of the current item,

- *the character is* **"put back"** *to be read again*
  - *during the scanning of the* **next input item** *(next* `%` *stuff) or*
  - *during the* **next call** *of* `scanf()` .

> The concept of pattern-matching can be taken one step further by writing format strings that contain ordinary characters in addition to conversion specifications. The action that scanf takes when it processes an ordinary character in a format string depends on whether or not it's a white-space character.

3 Type of Characters in **Format-String** or The Input **:**

1. 🟩 **Matching Characters /w Conv. Specs.**
   - *Well, we know what* `scanf()` *does to these ones* 🤷‍♀️
   - *Everything else other than* **matching chars** ➡️ **Ordinary Characters**
2. 🟩 **White-space characters**

- *Remember,* **white-space characters**
  - ` ` - ASCII 32 - **space**
  - `\n` - ASCII 10 - **newline** - *Moves cursor to the beginning of the next line*
  - `\t` - ASCII 9 - **horizontal tab**
  - `\v` - ASCII 11 - **vertical tab**
  - `\f` - ASCII 12 - **form feed** - *Advances the output to the next page or form*
  - `\r` - ASCII 13 - **carriage return** - *Moves cursor to the beginning of the current line*

- *When* `scanf()` *encounters one or more* **white-space characters** *in a* **format string**
  - `scanf()` *reads all* **white-space characters** *from the input until it reaches a* **non-white-space character.**
- *So, if* `scanf()` *finds*
  - `1` *Space or*
  - `1` *single* `\t` *or*
  - `1` *single* `\n`
    - *then, it will* **keep discarding all** *the* **white-space characters** *together*
    - *untill it finds a* **non-white-space**
- *This* **non-white-space** *is now gonna be compared with the next character in our* **format-string**
- 🎴 *Now,* `1` **white-space char.** *in the* **format-string** *will match any number of* **white-space char.** *in the input.*
- *But, putting a* **white-space char.** *in a* **format-string** doesn't force **the input** *to contain* **white-space char.**

3. 🟩 **Other characters** ➡️ `SMTH`
   - *When* `scanf()` *encounters a* **non-white-space** *character in a* **format string**
     - `scanf()` *compares it with the next input character*
       - *If they match* ➡️ *The* `scanf()` *discards the input character and continues processing* **format string**
       - *If they don't match* ➡️ `scanf()` **puts** *the offending character* **back into the input**, *then* **aborts**

- 🟩 *Examples:*

```
scanf("%d/%d", &x, &y);

input: ·5/·96
    x=5, y=96
    Everything Works Out!
```

```
scanf("%d/%d", &x, &y);

input: ·5·/·96
    scanf() fails
    the / in the "format-string" doesn't match the space in the input
```

  - *To allow spaces after the first number,*
    - *we should use the* **format-string** `"%d /%d"` *instead.*

3.2.16. Confusing `printf()` with `scanf()` 🟩 (unedited)

- *Don't use ordinary characters in* `scanf()` *unless it is necessary*

```c
scanf("%d, %d", &i, &j);    // Not error, but be careful
```

first, `scanf()` *will look for an* `int` *in the input* ➡️ *stores in* `i`

then, `scanf()` *takes the next character in the* **format-string**

- `,`
  - *This is not a* **1. Conv. Specs.**
  - *This is not a* **2. White-Space Character**
  - *This is a* **3. Ordinary Other Character**
- *Take a look back @ the* **3. Other Character** *Handling rules from last slide* 😉
- *If the next input character is a* ` ` *space, not a* `,` *comma*
  - *then* `scanf()` *realizes that they don't match*
  - *and as per the rules described in the last slide,*
    - `scanf()` *will terminate without reading a value for `j`.*

- *Don't use ampersand in* `printf()`

```c
printf("%d %d\n", &i,&j);   // Wrong
```

- *Don't put a* **new-line character** *at the end of a* `scanf()` **format-string**

```c
scanf("%d\n", &i);
scanf("%d \n", &i);
scanf("%d \n ", &i);
```

- *If the* **format-string** *is* `"%d\n"`,
  `scanf()` *will*
  - `%d`
    - *skip* **2. White-space characters**
    - *read an integer, by finding* **1. Matching characters**
      - *stop as soon as it hits a* **non-digit** ➡️ *let's call that* **3. Other characters**
  - *So this* **3. Other character** *is now gonna be compared with the next character in the* **format-string**, *which apparently is* `\n`
  - `\n`
    - *is itself a* **2. White-space characters**
    - *This* **trailing whitespace** *in our* **format-string** *tells* `scanf()` *to keep reading and discarding any and all* **2. whitespace**
  - *The Stopping Condition:*
    - `scanf()` *only finishes that trailing* `\n` *directive*
      when `scanf()` *encounters something that is not a* **2. whitespace** *(like a letter or another number)*
    - *So, if we enter* `50\n`
      - `scanf()` *won't end / won't* `return` *just yet*
      - *thus --> cause interactive programs to hang*

- *If the* **format-string** *is* `"%d \n"` ➡️ *Same would happen....*
  - *Because* ` ` *(space) &* `\n` *are both* **2. White-space characters**
- *If the* **format-string** *is* `"%d \n "` ➡️ *Same would happen....*

3.2.17. Program : `Adding Fractions`

```
Enter first fraction: 5/6
Enter second fraction: 3/4
The sum is 38/24
```

- *Consider a* `C` *program that prompts the user to enter two fractions and then displays their sum*
- *Fractions are customarily denoted as: numerator/denominator*

Solution

```c
#include <stdio.h>

int main(void)
{
    int n1, d1;
        printf("Enter first fraction: ");
        scanf("%d / %d");

    int n2, d2;
        printf("Enter second fraction: ");
        scanf("%d / %d");

    int lcm_d = find_lcm(d2, d2);
    int d = lcm_d;
    int n = (n1*(d/d1)) + (n2*(d/d2))

    printf("The sum is %d/%d", n, d);
}
```

```c
// Function to find GCD using Euclid's algorithm (Recursive)
int find_gcd(int a, int b) {
    if (b == 0)
        return a;
    return find_gcd(b, a % b);
}

// Function to find LCM using the GCD result
int find_lcm(int a, int b) {
    return (a * b) / find_gcd(a, b);
}
```

$$LCM(a,b) = \frac{|a \times b|}{\text{GCD}(a,b)}$$

> REYNEP's yap yap
>
> So, remember `get_int()` , `get_float()` , `get_string()` from week 1 (cs50)?
> These are sorta' like some really nice training wheels on a bicycle 🤷 that cs50 gives us.
>
> Now, even cs50 doesn't show `scanf()` untill before the middle of the course, cz `scanf()` is very sophisticated, one might even say that it is goddamn scary.
>
> cs50 introduces `scanf()` @ 2h01m of week 4, that's like the 12th hour of this course 🤷.
> and the course is 24hours altogether.
>
> `scanf()` also requires for the user to have knowledge about pointers, which is introduced in week 4.

The End

Chapter 4
Expressions

CSE 4107 : Structured Programming I
REYNEP

4.0. Expressions

One of *C* 's distinguishing characteristics is it's emphasis on **expressions**

- **expressions** *are formulas that show how to compute a value*
- **expressions** *are built from variables, constants & operators*

Operators are the basic tools for building expressions, and C has an unusually rich collection of them:

- **arithmetic** *operators*
- **relational** *operators*
- **logical** *operators*
- **assignment** *operators*
- **increment** *and* **decrement** *operators*

4.1. Arithmetic Operators

Binary (**two** operands) Arithmetic Operators:

```
addition                int x = 10 + 5;
subtraction             int x = 10 - 5;
multiplication          int x = 10 * 5;
division                int x = 10 / 5;
remainder = modulus     int x = 10 % 5;
```

Unary (**one** operands) Arithmetic Operators:

```
                        int y =  7;
unary plus              int x = +y;     int m = +1;
unary minus             int x = -y;     int n = -2;
```

The 'unary plus' emphasizes that the numeric constant is positive

Binary (**two** operands) Arithmetic Operators allow:

- **int** *operands*
- **float** *operands*
- *allow mixing of* `int` & `float`
  - *When* `int` *and* `float` *operands are* **mixed**, *the result has type* `float`
    - `9 + 2.5f` *has the value* `11.5`
    - `6.7f / 2` *has the value* `3.35`
- **remainder** *operator*
  - `int k = x % y;`
  - *The value of* `k` *is now the* **remainder** *when* `x` *is divided by* `y`
    - `10 % 3` *has the value* `1`
    - `12 % 4` *has the value* `0`
- *All the* **Binary Arithmetic** *Operators allow* `int` , `float` , *or even* **mixing**
  - *except for* **remainder** *operator* `%`

```
float a =    9 + 2.5f;      // a = 11.5
float b = 6.7f / 2;         // b = 3.35
  int c =   10 % 3;         // c = 1
  int d =   12 % 4;         // d = 0
```

```
int a =   1 / 2;            // a = 0
int b =   5 % 2;            // works
int c = 5.0 % 2;            // the program won't compile

int d = 6.0 / 0;
    // undefined behavior

int e =  -5 /  2;
int e =   5 / -2;
    // C89: '-2' or '-3'     🤷 varies from compiler to compiler
    // C99: '-2' always         truncated towards zero.

int f =  -5 %  2;
int f =   5 % -2;
    // C89: varies from compiler to compiler
    // C99: has the same sign as 'left-operand'
```

Ivans lines

The `/` and `%` operators require special care:

- *When both operands are* `int`
  - `int x = 1/2;` ➡️ `x = 0` *not* `0.5`
  - `/` *"truncates" the result.*
- *The* `%` *operator requires* `int` *operands*
  - *if either operand is not an* `int` *, the program won't compile*
- *Using* `0` *zero as the right operand of either* `/` *or* `%` *causes undefined behavior*
- *negative operands*
  - `C89` *: The behavior when* `/` *and* `%` *are used with negative operands is* **implementation-defined** *(see next page)*
  - `C99` *: the result of a division is always truncated toward* `0` *zero and the value of* `x % y` *has the same sign as* `x`

4.1. Glossary

The *C* Standard

- *Every language has a* **"Manual"** *or a* **"Guide"** *that all it's* **compilers** *must follow*
- *It's better to call it* **The C Programming Language Compiler Specifications**
  - *or,* **The C Compiler Specs.**
  - *Now, this* **Specs.** *are made by* **ISO** *(International Organization for Standardization)*
    - *Since, what they do is,* **"to Standardize"**
    - *They name stuff* **"... Standard"**
    - *in our case,* **"The C Standard"**
- *old people just likes to call it* **The *C* Standard**

Implementation - what does it mean?

- *different compilers are* **made** *diffrently*
- *different compilers are* **implemented** *diffrently*

🟩 **implementation-defined** behavior

- **The *C* standard** *deliberately leaves parts of the language,* **unspecified**
  - *However, this does, reflect* ***C****'s emphasis on efficiency,*
    - *which often means matching the way that hardware behaves*
    - *e.g. in* `C89`
      - `-9 / 7 = -1` *it could be this*
      - `-9 / 7 = -2` *it could be that*
      - 🤷‍♀️ *we don't know*
    - *So, back in those days, people would have to "know" about the hardware differences & write different variations of code for different hardwares*
- *So, different compilers will handle this* **unspecified** *part, differently.*
  - *So it varies as per* **compiler-implementation**
  - *So it's* **compiler-implementation-defined**
  - *So it's* **implementation-defined**
- 🧧 *It's best to* **avoid** *writing programs that depend on* **implementation-defined** *behavior*

Operator Precedence

`i+j*k`

- *is it* `(i+j) * k` *? or*
- *is it* `i + (j*k)` *?*

- *we should kind of, always just use parentheses*
- *and if parentheses are omitted,* ***C*** *uses* **operator precedence** *rules to determine the meaning of the expression*

```
(Highest)   Level-1:        + -     (unary)
            Level-2:        * / %   (binary)
(Lowest)    Level-3:        + -     (binary)
```

```
 i +  j * k        is equivalent to          i  + ( j * k)
-i * -j            is equivalent to        (-i) * (-j)
+i +  j / k        is equivalent to        (+i) + ( j / k)
```

- *When in doubt, use this table or just use* **plenty of parentheses**

Operator Associativity

```
i - j - k       is equivalent to        (i - j) - k
i * j / k       is equivalent to        (i * j) / k
```

- **Associativity** *comes into play when an expression contains two or more operators with equal precedence*
- *The* **binary arithmetic** *operators (*, /, %, +, and -) are all* **left associative**
  - *An operator is said to be* **left associative** *if it groups from left to right*
- *The* **unary arithmetic** *operators (+ and -) are both* **right associative**
  - *An operator is* **right associative** *if it groups from right to left*

4.1.2. Program : `Computing a UPC Check Digit`

For a number of years, manufacturers of goods sold in U.S. and Canadian stores have put a bar code on each product.

- *This code, known as a Universal Product Code (UPC), identifies both the manufacturer and the product.*
- *Each bar code represents a twelve-digit number*
  - *which is usually printed underneath the bars.*
  - *For example,*
    - *the following bar code comes from a package of* **Stouffer's French Bread Pepperoni Pizza**:



    - *The digits*

```
0 13800 15173 5
```

appear underneath the bar code.

      - *The first digit identifies the type of item*
        - *0 or 7 for most items,*
        - *2 for items that must be weighed,*
        - *3 for drugs and health-related merchandise, and*
        - *5 for coupons.*
      - *The first group (13800) of five digits identifies the manufacturer*
        - *(13800 is the code for Nestlé USA's Frozen Food Division).*
      - *The second group of five digits identifies the product (including package size).*
      - *The final digit is a "check digit," whose only purpose is to help identify an error in the preceding digits.*
        - *If the UPC is scanned incorrectly, the first 11 digits probably won't be consistent with the last digit, and the store's scanner will reject the entire code.*

Here's one method of computing the check digit:

1. *Add the Odd-Indexed Digits* ➡️ `sum1` *- i.e. first, third, fifth, seventh, ninth, and eleventh digits.*
2. *Add the Even-Indexed* ➡️ `sum2` *- i.e. second, fourth, sixth, eighth, and tenth digits.*
3. *Multiply* `sum1` *by 3 and add it to* `sum2` ➡️ `total`
4. *Subtract 1 from the* `total` .
5. *Compute the remainder when the adjusted total is divided by 10.*
6. *Subtract the remainder from 9.*

Using the Stouffer's example

1. *we get 0 + 3 + 0 + 1 + 1 + 3 = 8* ➡️ `sum1`
2. *1 + 8 + 0 + 5 + 7 = 21* ➡️ `sum2`
3. *Multiplying* `sum1` *by 3 and adding* `sum2` *yields 45.* ➡️ `total`
4. *Subtracting 1 from* `total` *gives 44.*
5. *The remainder upon dividing* `total` *by 10 is 4.*
6. *When the remainder is subtracted from 9, the result is 5.*

Here are a couple of other UPCs, in case you want to try your hand at computing the check digit

(raiding the kitchen cabinet for the answer is not allowed).
(The answers appear at the bottom of the page.)

```
          Jif Creamy Peanut Butter (18 oz.):    0 51500 24128 ?
Ocean Spray Jellied Cranberry Sauce (8 oz.):    0 31200 01005 ?
```

THE PROBLEM: Let's write a program that calculates the check digit for an arbitrary UPC.

- *We'll ask the user to enter the first 11 digits of the UPC,*
- *then we'll display the corresponding check digit.*
- *To avoid confusion, we'll ask the user to enter the number in three parts:*
  - *the single digit at the left,*
  - *the first group of five digits,*
  - *and the second group of five digits.*
- *Here's what a session with the program will look like:*

```
Enter the first (single) digit: 0
Enter first group of five digits: 13800
Enter second group of five digits: 15173
Check digit: 5
```

  - *Instead of reading*
    - *each digit group as a* **"five-digit number"**,
  - *we'll read it as*
    - *five* **"one-digit numbers"**.
  - *Reading the numbers as single digits is more convenient;*
  - *also, we won't have to worry that*
    - *maybe one of the five-digit numbers is too large to store in an* `int` *variable.*
      - *i.e. Compilers 30years ago, in 1980s, used to see* `int` *as* **16-bit**
      - *So, The Author Wrote This:*
        - *(Some older compilers limit the maximum value of an int variable to* `32,767` *).*
  - *To read single digits, we'll use* `scanf()` *with the* `%1d` **conversion specification**,
    - `%1d` *matches a one-digit integer*

The Solution:

Recall back to this:

Here's one method of computing the check digit:

1. *Add the Odd-Indexed Digits* ➡️ `sum1` *- i.e. first, third, fifth, seventh, ninth, and eleventh digits.*
2. *Add the Even-Indexed* ➡️ `sum2` *- i.e. second, fourth, sixth, eighth, and tenth digits.*
3. *Multiply* `sum1` *by 3 and add it to* `sum2` ➡️ `total`
4. *Subtract 1 from the* `total` *.*
5. *Compute the remainder when the adjusted total is divided by 10.*
6. *Subtract the remainder from 9.*

Using the Stouffer's example

1. *we get 0 + 3 + 0 + 1 + 1 + 3 = 8* ➡️ `sum1`
2. *1 + 8 + 0 + 5 + 7 = 21* ➡️ `sum2`
3. *Multiplying* `sum1` *by 3 and adding* `sum2` *yields 45.* ➡️ `total`
4. *Subtracting 1 from* `total` *gives 44.*
5. *The remainder upon dividing* `total` *by 10 is 4.*
6. *When the remainder is subtracted from 9, the result is 5.*

Now we implement:

```c
/* Computes a Universal Product Code check digit */
#include <stdio.h>

int main(void)
{
    int d;
        printf("Enter the first (single) digit: ");
        scanf("%1d", &d);

    int i1, i2, i3, i4, i5;
        printf("Enter first group of five digits: ");
        scanf("%1d%1d%1d%1d%1d", &i1, &i2, &i3, &i4, &i5);

    int j6, j7, j8, j9, j10;
        printf("Enter second group of five digits: ");
        scanf("%1d%1d%1d%1d%1d", &j6, &j7, &j8, &j9, &j10);

    int sum1  = d + i2 + i4 + j6 + j8 + j10;
    int sum2  =     i1 + i3 + i5 + j7 + j9;
    int total = 3*sum1 + sum2;


    int ans   = 9 - ((total - 1) % 10);

    printf("Check digit: %d\n", ans);
    return 0;
}
```

- *Note that the expression* `9 - ((total - 1) % 10)` *could have been written as* `9 - (total - 1) % 10` *,*
    - *but the extra set of parentheses makes it easier to understand.*

4.2. Assignment = Operator ? & lvalue

- **Simple** *Assignment* 🟩 *used for storing a value into a variable*

```
i = 5;
j = i;
k = 10*i + j;
```

```
int i;      float f;
i = 72.99f;
f = 136;
```

  - *The effect of the assignment* `v = e`
  - *is to evaluate the expression* `e` *and copy it's value into* `v`
    - `e` := **Expression**
      - *can be a* **constant**
      - *can be a* **variable**
      - *can be a* **more complicated expression**
  - **If** `v` *and* `e` *don't have the same type,* **then** *the value of* `e` *is converted to the type of* `v` *as the assignment takes place:*

- **Simple** *Assignment :* **SideEffects** 🟩

```
line 1:     k = k+ i=72.99f;                // many languages won't allow this
line 2:     k = k+   72.99f;   i=72.99f;    // But C does & both lines are same in C
```

  - *In* C, **assignment** *is an operator, just like* `+`
  - *The value of* `v=e` $\implies$ *the value of* `v` *after the assignment*
    - `int i` $\implies$ *The value of* `i = 72.99f` *is* `72`
  - 🟦 **assignment** *being an operator*
    - *Simply means that,* `line 1` *in the above example is 'do-able'*
    - *However, In many programming languages, it is not 'do-able'*
      - *in those programming languages* **assignment** *is a statement.*
  - *Since* **assignment** *is an operator in* C,
    - *several assignments can be chained together:*

```
int i, j, k;
i = j = k = 0;
```

    - *The* `=` *operator is* **right associative**, *so this "chain of assignments" is equivalent to*

```
int i, j, k;
i = (j = (k = 0));
```

  - *An operator that modifies one of its operands is said to have a* **side effect**
  - *The* **simple assignment** *operator has a* **side effect**
    - *it modifies its left operand*
  - *Evaluating the expression* `i = 0` *produces the result* `0` *and—as a side effect—assigns* `0` *to* `i`
  - *Wath out of unexpected results in chained assignments as a result of type conversion:*

```
int i;
float f;

f = i = 33.3f;
```

    - `i` *is assigned the value* `33`, *then* `f = 33` *(not* 33.3*)*

```
int i = 1;
int k = 1 + (j = i);

printf("%d %d %d\n", i, j, k);          /* prints "1 1 2" */
```

    - *An assignment of the form* `j = i` *is allowed wherever a value of type* `j` *would be permitted:*
    - **"Embedded assignments"** *can make programs hard to read.*
    - *They can also be a source of* **subtle bugs**.

- **Simple** Assignment : **lvalue** 🟩
  - **lvalue** *Represents an* **object/locker** *stored in computer memory,*

```
2 = i;          /*** WRONG ***/
i + j = 0;      /*** WRONG ***/
-i = j;         /*** WRONG ***/
```

    - *The compiler will produce an error message such as* **"invalid lvalue in assignment"**
    - *So, one could say,*
    - **Variables** *are* **lvalues**
    - **constants** *are* **not**, **expressions** *are* **not**
  - **lvalue** *does not represent*
    - *a constant or*
    - *the result of a computation (which is also another constant)*
  - > **Left operand** *of a assignment always has to be an* **lvalue**

- **Compound Assignment** 🟩

```
i += 6      //      i = i + 2;
i -= 5      //      i = i - 2;
i *= 4      //      i = i * 2;
i /= 3      //      i = i / 2;
i %= 2      //      i = i % 2;
```

```
i *=     j+k;
i  = i * j+k;       // These 2 are not the same

i *=     j+k;
i  = i *(j+k);      // However these 2 are same.
```

  - *All* **compound** *assignment ops work in the same way:*
    - *These are assignments that use the old value of a variable to compute its new value & very common*
    - *There are 9 of them, including the following:*
      - `-= *= /= %=`
  - `i =+ j` $\implies$ `i = (+j)`
    - *which merely copies the value of* `j` *into* `i`
    - *So, when using them* `+=`, *be careful not to switch the two characters that make up the operator.*
  - *So,* **ivan** *says*
    - `v += e` **isn't "equivalent"** *to* `v = v+e`
      - *he really should have clarified, that he means* **"multiple ops"** *inside* `e`
    - *One problem is operator precedence:*
      - *as in the example above*
    - *There are also different rathermore rare cases in which*
      - `v += e` *differs from* `v = v+e` *because* `v` *itself has a side effect.*
      - *Similar remarks apply to the other compound assignment operators*

4.3. Increment & Decrement Operators

Two of the most common operations on a variable are

```
i = i + 1;
j = j - 1;
```

1. **“incrementing”** *(adding* `1` *)*
2. **“decrementing”** *(subtracting* `1` *)*

```
i += 1;     // 1. incrementing
j -= 1;     // 2. decrementing
```

- *can be also done using the* **Compound** *assignment operators, like above*

```
i++;        // i += 1
j++;        // j += 1
```

```
++i;        // another way
++j;        // another way
```

ivan's lines

- *C provides special* `++` *(increment) and* `--` *(decrement) operators.*
- *The* `++` *operator adds 1 to its operand. The* `--` *operator subtracts 1.*
- *The* **increment** *and* **decrement** *operators are tricky to use:*
  - *They can be used as*
    - **prefix** *operators (* `++i` *and* `--i` *) or*
    - **postfix** *operators (* `i++` *and* `i--` *)*
  - *They have* **side effects**:
    - *they modify the values of their operands*

postfix vs prefix operators

`++i` **(a “pre-increment”)**

```
int i = 1;
printf("i is %d\n", ++i);   /* prints "i is 2" */
printf("i is %d\n", i);     /* prints "i is 2" */

int i = 1;
printf("i is %d\n", --i);   /* prints "i is 0" */
printf("i is %d\n", i);     /* prints "i is 0" */
```

- `++i` $\implies$ **increment** `i` **immediately**
- *Evaluating the expression* `++i`
  - *yields* `i+1` *and as a side effect* $\longrightarrow$ *increments*

`i++` **(a “post-increment”)**

```
int i = 1;
printf("i is %d\n", i++);   /* prints "i is 1" */
printf("i is %d\n", i);     /* prints "i is 2" */

int i = 1;
printf("i is %d\n", i--);   /* prints "i is 1" */
printf("i is %d\n", i);     /* prints "i is 0" */
```

- `i++` $\implies$ **use the old value** *of* `i` *for now, but* **increment** `i` **later**
- *Evaluating the expression* `i++`
  - *produces the result* `i` *, but causes* `i` *to be incremented afterwards*
- *How much later?*
  - *The C standard doesn't specify a precise time, but*
    - *it's safe to assume that i will be incremented before the next statement is executed*

When `++` or `--` is used more than once in the same expression, the result can often be hard to understand.

```
int i = 1;
int j = 2;
int k = ++i + j++;
```

- *This statement is equivalent to the on on the right*

```
int i = 1;
int j = 2;
    i = i + 1;      // i = 1+1
    k = i + j;      // k = 2+2
    j = j + 1;      // j = 2+1
```

```
int i = 1;
int j = 2;
int k = i++ + j++;
```

- *both are* **postfix ops** *here*

```
    .
    .
    k = i+j;    i+=1;     j+=1;
```

- *This is the breakdown of the one on the left*

4.4. Expression Evaluation

Updated Precedence Table

Precedence	Name	Symbol	Associativity
1 (Highest)	increment (postfix)	`i++`	left
	decrement (postfix)	`i--`	
2	increment (prefix)	`++i`	right
	decrement (prefix)	`--i`	
	unary plus	`+x`	
	unary minus	`-x`	
3	multiplicative	`*` `%`	left
4	additive	`+` `-`	left
5 (lowest)	assignment	`= *= /= %= += -=`	right

```
                                                Precedence level
 a =  b +=    c++  - d  +   --e  /  -f
 a =  b +=   (c++) - d  +   --e  /  -f                  1
 a =  b +=   (c++) - d  +  (--e) / (-f)                 2
 a =  b +=   (c++) - d  + ((--e) / (-f))                3
 a =  b += (((c++) - d) + ((--e) / (-f)))               4
(a = (b += (((c++) - d) + ((--e) / (-f)))))             5
```

The table can be used to add parentheses to an expression that lacks them.

Starting with the operator with highest precedence put parentheses around the operator and its operands.

Order of Subexpression Evaluation

- *C does define the order for these subexpressions*
  - `&&`
  - `||`
  - *lines that have comma operators* $\Longrightarrow$ `int a = 0, b = 1;`
  - *conditional*

```c
int a, b, c;
a = 5;
c = (b = a+2) - (a = 1);      // Undefined behavior
```

The value of an expression may depend on the order in which its subexpressions are evaluated

- *C doesn't define the order in which subexpressions are evaluated, normally*
  - *Is it* `(a = 1)` *first?*
  - *Is it* `(b = a+2)` *first?*
  - *Say for the the expression,* `(a+b) * (c-d)`
    - *we don't know whether* `(a+b)` *will be evaluated before* `(c-d)`
  - > *Most expressions have the same value regardless of the order in which their subexpressions are evaluated*
    > *However, this may not be true when a subexpression modifies one of its operands:*

    ```c
    int a, b, c;
    a = 5;
    c = (b = a+2) - (a = 1);      // Undefined behavior
    ```

    - *The effect of executing the second statement is undefined*
  - *We could use a series of separate assignments instead:*

    ```c
    a = 5;
    b = a + 2;
    a = 1;
    c = b - a; // The value of c will always be 6
    ```

    - *Avoid writing expressions that access the value of a variable and also modify the variable elsewhere in the expression*
      - *Some compilers may produce a warning message, if you do so....*
    - *It's a good idea to avoid using the assignment operators in subexpressions.*

- *Besides the assignment operators, the only operators that modify their operands are increment and decrement.*

```c
i = 2;
j = i * i++;
```

  - *It's natural to assume that* `j` *is assigned* `4`*. However,* `j` *could just as well be assigned* `6` *instead:*
    a. *The second operand (the original value of* `i`*) is fetched, then* `i` *is incremented*
    b. *The first operand (the new value of* `i`*) is fetched*
    c. *The new and old values of* `i` *are multiplied, yielding* `6`
  - *When using these operators, be careful that an expression doesn't depend on a particular order of evaluation*

Undefined Behavior

- *Statements such as* `c = (b = a + 2) - (a = 1);` *and* `j = i * i++;` *cause undefined behavior*
- *Possible effects of undefined behavior:*
  - *The program may* **behave differently** *when compiled with different compilers*
  - *The program* **may not compile** *in the first place*
  - *The program may compile, and, If it compiles it* **may not run**
  - *The program may compile & run, and, If it does run, the program* **may crash, behave erratically**, *or produce meaningless results*

- *Code that leads to* **Undefined behavior**, *should be avoided*

4.5. Expression Statements

- *Expressions can be discarded.*
  - *So there's little point in using an expression as a statement unless the expression has a side effect:*

```
i = 1;          /* useful */
i--;            /* useful */
i * j - 1;      /* not useful */
```

    - *by "side effect", author meant, an expression that changes some values in memory*

  - *Expressions can be used as a statement by appending ';'*

- *A slip of the finger can easily create a "do-nothing" expression statement*
- *For example,*
  - *instead of entering* `i = j`
  - *we might accidentally type* `i + j`

- *Some compilers can detect meaningless expression statements;*
  - *you'll get awarning such as "statement with no effect."*

The End

Chapter 5
Selection Statements

CSE 4107 : Structured Programming I
REYNEP

5.0. Statements

So far, we've used
- *expression statements*
- *return statements*

Other C statements:
- **Compound** *statements*
- *null statements*

Most of C's remaining statements fall into three categories:
- **Selection** *statements:* `if` `switch`
- **Iteration** *statements:* `while` `do` `for`
- **Jump** *statements:* `break` `continue` `goto` `return`

5.1.1. boolean Expressions

```c
int main(void) {
    int i = 0, j = 20;

    if (i < j) {
        printf(" true: i is less than j");
    }
    else {
        printf("false: i is not less than j");
    }
}
```

- *A* **true** *value would indicate that i is less than j.*

`if`
- *tests the value of an expression to see if it's* **"true"**
  - *being* **true** $\implies$ *processor gets inside the* `if` *-block*
  - *being* **false** $\implies$ *processor gets inside the* `else` *-block*

```c
if (true) {
    printf("We are inside the true block");
}
```

- *in C, the words* `true` `false` *are inside a file called* `stdbool.h`

```c
#include <stdio.h>


int main(void)
{
    if (1) {
        printf(" true: ");
    }
    else {
        printf("false: ");
    }
}
```

```c
#include <stdio.h>
#include <stdbool.h>

int main(void)
{                   /* stdbool.h --> words: 'true', 'false' */
    if (true) {
        printf(" true: this will be printed");
    }
    else {
        printf("false: this won't be printed");
    }
}
```

- *in C*
  - `1` $=$ `true`
  - `0` $=$ `false`
  - `if (i < j)` $\implies$ `if (1)` *for our example case above*
    - *bcz,* `i=0` , `j=20`

- *in fact, inside* `stdbool.h` *, you can find*
  - `#define true 1`
  - `#define false 0`

-
  - *it's kinda like*
    - *the C compiler's dictionary has the words say* `int` `float` `double` `char` `void` , `return` , `if` `else`
    - *the* `stdio.h` *file has the words/definitions of say* `printf()` , `scanf()`
    - *the C compiler's dictionary doesn't have the word* `true` `false`
    - *the* `stdbool.h` *file adds these words*

Let me firmly, introduce another new data-type → `bool`

```c
#include <stdio.h>
#include <stdbool.h>

int main(void)
{
    bool result = isPrime(50);

    if (result == true) {
        printf("50 is prime");
    }
    else {
        printf("50 is not prime");
    }
}
```

- *Other programming languages* `python` `java` `C++`
  - *has* `bool` *as* **built-in data type**
- *But in C*
  - `stdbool.h` **defines** *it*
    - *However, the story behind "How it's def." is kinda long*
      - *https://stackoverflow.com/a/47822105*
- *in those other programming languages,*
  - *an expression such as* `i<j` *would have a* `bool` */logical data-type*
- *But in* `C`
  - *such comparisons yield an integer i.e.* `0` *or* `1`

5.1.2. Operators

Binary (two operands) Arithmetic Operators:

```
addition                int x = 10 + 5;
subtraction             int x = 10 - 5;
multiplication          int x = 10 * 5;
division                int x = 10 / 5;
remainder = modulus     int x = 10 % 5;
```

```
float a =    9 + 2.5f;      // a = 11.5
float b = 6.7f / 2;         // b = 3.35
  int c =   10 % 3;         // c = 1
  int d =   12 % 4;         // d = 0
```

Unary (one operands) Arithmetic Operators:

```
                    int y =  7;
unary plus          int x = +y;     int m = +1;
unary minus         int x = -y;     int n = -2;
```

Simple Assignment Operator

```
i = 5;
j = i;
k = 10*i + j;
```

```
int i, j, k;
i = j = k = 0;
```

[in/de]crement operators

```
i++;
i--;
```

- **post-[in/de]crement**

```
++i;
--i;
```

- **pre-[in/de]crement**

Compound Assignment Operator

```
i += 6      //      i = i + 2;
i -= 5      //      i = i - 2;
i *= 4      //      i = i * 2;
i /= 3      //      i = i / 2;
i %= 2      //      i = i % 2;
```

Relational Operators

```
5 < 10          // true
5 > 10          // false
101 <= 102      // true
101 >= 102      // false
```

Equality Operators

```
101 == 101      // true
101 != 101      // false
```

ivan's @ page 04

- *These operators produce* `0` *(false) or* `1` *(true) when used in expressions, say inside* `if ()`
- *These operators can be used to compare*
  - *integers*
  - *floating-point numbers,*
  - *operands of mixed types*
- *The* **precedence** *of the* **relational** *operators is lower than that of the* **arithmetic** *operators*
  - *e.g.* `i + j < k - 1` $\implies$ `(i + j) < (k - 1)`

ivan's @ page 05

- *The* **relational** *operators are left associative*

```
i < j < k
```

  - *is legal*
  - *but, does not test, whether* `j` **lies between** `i` **and** `k`

```
(i < j) < k
```

  - *Since the* `<` *operator is left associative, it ends up like this, above.*
    - *The 1 or 0 produced by* `i < j` *is then compared to* `k`

- `j` **lies between** `i` **and** `k`
  - *The correct expression is* `i<j && j<k`

- *The* **Equality** *Operators*
  - *are also left associative*
  - *also produce either 0 (false) or 1 (true) as their result*
  - *have* **lower precedence** *than the* **relational** *operators*
    - *So, The expression*
      - `i < j == j < k` *is equivalent to,* `(i < j) == (j < k)`

Logical Operators

```
bool a = true;
bool b = !a;        // false;
bool c = a && b;    // false;
bool d = a || b;    // true;
```

ivan's @ page 08

- *The* `!` *operator is* **unary**
- `&&` *and* `||` *operators are* **binary**.
- *The* **logical** *operators*
  - *produce* `0` *or* `1` *as their result*
  - *treat any non-zero operand as a* `true` *value and any zero operand as a* `false` *value*

ivan's @ page 09

- *Behavior of the logical operators:*
  - `!expr` *has the value* `1` *if* `expr` *has the value* `0`
  - `expr1 && expr2` *has the value* `1` *if the values of* `expr1` *and* `expr2` *are both nonzero*
  - `expr1 || expr2` *has the value* `1` *if either* `expr1` *or* `expr2` *(or both) has a nonzero*
  - *In all other cases, these operators produce the value 0*

"short-circuit" evaluation

- `&&` `||`
  - *first evaluate the* **left** *operand,*
  - *then the* **right** *one*

```
if ((i != 0) && (j/i > 0))    { printf("we do something in this block"); }
```

- *If the value of the expression can be deduced from the left operand alone, the right operand isn't evaluated*
  - `(i != 0)` *is evaluated first.*
    - **If,** *this part is* **true,** *only then the right part is checked. (notice we used* `&&` *)*
    - **If,** *this part is* **false,** *the entire expression must be false, so there's no need to evaluate* `(j/i > 0)`
      - *Without short-circuit evaluation, division by zero would have occurred, in this case*

"short-circuit" evaluation (continued)

```
if (i > 0 && ++j > 0)
{

}
```

- *If* `i > 0` *is* `false`*, then* `++j > 0` *is not evaluated, so* `j` *isn't incremented*
- *Side effects in logical expressions may not always occur because of the short-circuit nature*
- *The problem can be fixed by changing the condition to*

```
if (++j > 0 && i > 0) {}
```

- *or, even better, by incrementing j separately*

ivan's @ page12 : Precedence

- *The* `!` *operator*
  - *has the same precedence as the* **unary** *plus and minus operators*
  - *is right associative*
- *The* `&&` *and* `||` *operators*
  - `&&` *and* `||` *are left associative*
  - *The precedence of* `&&` *and* `||` *is lower than that of the relational and equality operators*

```
i < j && k == m
```
$\Longrightarrow$
```
(i < j) && (k == m)
```

5.1.3. Operators : all at once

Assignment Operators

Simple

```
i = 5;
j = i;
k = 10*i + j;
```

```
int i, j, k;
i = j = k = 0;
```

Compound

```
i += 6      //      i = i + 2;
i -= 5      //      i = i - 2;
i *= 4      //      i = i * 2;
i /= 3      //      i = i / 2;
i %= 2      //      i = i % 2;
```

```
i++;
i--;
```

- **post**

```
++i;
--i;
```

- **pre**

Boolean Operators

Relational Operators

```
5 < 10        // true
5 > 10        // false
101 <= 102    // true
101 >= 102    // false
```

Logical Operators

```
bool a = true;
bool b = !a;        // false;
bool c = a && b;    // false;
bool d = a || b;    // true;
```

Equality Operators

```
101 == 101    // true
101 != 101    // false
```

5.2.1. The `if` Statement

In its simplest form, the if statement has the form:

```
//
if (expression) {
    statements;
}
```

```
bool b = isPrime(50);
if (b == false) {
    printf("50 is prime");
}
```

- *When an* `if` *statement is executed,*
  - **expression** *is evaluated;*
    - `If` *that's value is nonzero,*
      - *then* **statements** *inside the curly barces are executed*

Possible Confusion

```
int  x = 50;            // 50;
bool b = (x == 50);     // true
int  b = (x == 50);     // 1
```

- *Notice that,* `==` *and* `=` *are different*
- *Confusing* `==` *(equality) with* `=` *(assignment) is perhaps the most common* C *prog. error*

```
if(i == 0) {}
```

- *tests whether* `i` $=$ `0`

```
if(i = 0) {}
```

- *assigns* `0` *to* `i`*, then tests whether the result is nonzero*

ivan's @ page15 : Multiple Statements

```
if (line_num == MAX_LINE_NUM)
{
    page_num++;
    line_num = 0;
}
```

- **ivan** *calls this* **Compound** *Statements*
- *A* **compound** *statement has the form* `{ statements }`
  - *A* **compound** *statement is usually put on multiple lines, with one statement per line:*
  - *Each inner statement still ends with a semicolon, but the compound statement itself does not*
- *To make an* `if` *statement "control" two or more statements, use a* **compound** *statement*
  - *by "control",* **ivan** *simply means here, "to execute"*

5.2.2. The `if` Statement (continued)

The `else` clause

```c
#include <stdio.h>
#include <stdbool.h>

int main(void)
{
    bool result = isPrime(50);

    if (result == true) {
        printf("50 is prime");
    }
    else {
        printf("50 is not prime");
    }
}
```

- *An* `if` *statement may have an* `else` *clause:*

```c
if ( expression ) statement_1 else statement_2
```

  - *statement_2 is executed* **when** *the* **expression** *has the value* `0`

```c
if (i > j) {
    max = i;
}
else {
    max = j;
}
```

```c
if (i > j)
    max = i;
else
    max = j;
```

```c
if (i > j) max = i;
else       max = j;
```

    - *all 3 are same*

Nested `if` Statements

```c
if (i > j) {        // ppl usually do many braces
    if (i > k)
        max = i;
    else
        max = k;
}
else {              // more braces ==> more safe
    if (j > k)
        max = j;
    else
        max = k;
}
```

```c
if (i > j)          // Can do this without Braces too
    if (i > k)
        max = i;
    else
        max = k;

else                // Indentation makes it much more readable
    if (j > k)
        max = j;
    else
        max = k;
```

Cascaded `if` Statement

```c
if (n < 0)
    printf("n is less than 0\n");
else
    if (n == 0)
        printf("n is equal to 0\n");
    else
        printf("n is greater than 0\n");
```

```c
if (n < 0)
    printf("n is less than 0\n");

else if (n == 0)
    printf("n is equal to 0\n");
else
    printf("n is greater than 0\n");
```

- *A* **"cascaded"** `if` *statement is often the best way to test a series of conditions*
  - *stopping as soon as one of them is true.*

- *Although the second* `if` *statement is* **nested** *inside the first, C programmers don't usually indent it.*
  - *Instead, they align each* `else` *with the original* `if`*, like the code on the right*

5.2.3. Program : `Calculating a Broker's Comission`

When stocks are sold or purchased through a broker

- *the broker's commission is often computed using a sliding scale that depends upon the value of the stocks traded.*
- *Let's say that a broker charges the amounts shown in the following table:*

Transaction size	*Commission rate*
Under $2,500	*$30 + 1.7%*
$2,500 – $6,250	*$56 + 0.66%*
$6,250 – $20,000	*$76 + 0.34%*
$20,000 – $50,000	*$100 + 0.22%*
$50,000 – $500,000	*$155 + 0.11%*
Over $500,000	*$255 + 0.09%*

  - *The minimum charge is $39.*
  - *Our next program asks the user to enter the amount of the trade, then displays the amount of the commission:*

```
Enter value of trade: 30000
Commission: $166.00
```

    - *The heart of the program is a* **cascaded** `if` *statement*
      - *that determines which range the trade falls into.*

5.2.3. Program : `Calculating a Broker's Comission` 🟩 ANSWER

```c
/* broker.c */
#include <stdio.h>
int main(void)
{
    float value;
        printf("Enter value of trade: ");
        scanf("%f", &value);

    float comission;
         if (value < 2500.00f)                              commission = 30.00f  + .017f  * value;
    else if ((value >= 2500.00f)  && (value < 6250.00f))    commission = 56.00f  + .0066f * value;
    else if ((value >= 6250.00f)  && (value < 20000.00f))   commission = 76.00f  + .0034f * value;
    else if ((value >= 20000.00f) && (value < 50000.00f))   commission = 100.00f + .0022f * value;
    else if ((value >= 50000.00f) && (value < 500000.00f))  commission = 155.00f + .0011f * value;
    else                                                    commission = 255.00f + .0009f * value;

    if (commission < 39.00f)                                commission = 39.00f;

    printf("Commission: $%.2f\n", commission);
}
```

The cascaded `if` statement could have been written this way instead:

```c
/* broker.c */
#include <stdio.h>
int main(void)
{
    float value;
        printf("Enter value of trade: ");
        scanf("%f", &value);

    float comission;
         if (value < 2500.00)       commission = 30.00  +  .017 * value;
    else if (value < 6250.00)       commission = 56.00  + .0066 * value;
    else if (value < 20000.00)      commission = 76.00  + .0034 * value;
    else if (value < 50000.00)      commission = 100.00 + .0022 * value;
    else if (value < 500000.00)     commission = 155.00 + .0011 * value;
    else                            commission = 255.00 + .0009 * value;

    if (commission < 39.00)         commission = 39.00;

    printf("Commission: $%.2f\n", commission);
}
```

5.2.4. 🟩 The "Dangling else" Problem

```c
if (y != 0)         // (1)
    if (x != 0)
        result = x / y;
else
    printf("Error: y is equal to 0\n");
```

- *The indentation suggests that the `else` clause belongs to*
  - *the* **outer** `if` *statement, i.e. belongs to* `(1)`
- *However, C follows the rule that an `else` clause belongs to*
  - *the* **nearest** `if` *statement that hasn't already been paired with an* `else`

- *This is called The* **"Dangling else"** *Problem*
  - *When* `if` *statements are nested, the "dangling else" problem may occur:*

The above code acts like:

```c
if (y != 0)
    if (x != 0)
        result = x / y;
    else
        printf("Error: y is equal to 0\n")
```

A correctly indented version would look like this:

```c
if (y != 0)
    if (x != 0)
        result = x / y;
    else
        printf("Error: x is equal to 0\n");
else
    printf("Error: y is equal to 0\n");
```

🟦 Or, the most ideal job to do, is to **"use more curly-braces"**

```c
if (y != 0) {
    if (x != 0)
        result = x / y;
}
else {
    printf("Error: y is equal to 0\n");
}
```

- *This would work as one would have expected in the first example right above, solely based on indentation*

5.2.5. Conditional/Ternary Operator

```c
#include <stdio.h>
#include <stdbool.h>

int main(void) {
    int i = 0, j = 20;



    if (i < j) {
        printf(" true: i is less than j");
    }
    else {
        printf("false: i is not less than j");
    }
}
```

```c
#include <stdio.h>
#include <stdbool.h>

int main(void) {
    int  i = 0, j = 20;

    bool b = (i < 50) ? true : false;       // Conditional Operator

    if (b == true) {
        printf(" true: i is less than j");
    }
    else {
        printf("false: i is not less than j");
    }
}
```

```c
int x = (condition_expr) ?  expr1_TRUE : expr2_FALSE;                 // yes, expr1/2 could be some random int 101

int x = (condition_expr)           ?  expr1_TRUE    : expr2_FALSE;
    if (condition_expr == true) then expr1_TRUE else expr2_FALSE;      // This is how you read it
```

- *The 3 operands*
  - `condition_expr`
  - `expr1_TRUE`
  - `expr2_FALSE`
  - *can be of any type*
- *C's* **conditional** *operator allows an* **expression** *to produce*
  - *one of two values (* `left` `right` *) of the colon* `:`
  - *depending on the value of the conditon*
- *The resulting expression is said to be a* **conditional** *expression*

---

ivan's @ page27

- *Because The* **conditional** *operator requires 3 operands,*
  - *it is often referred to as the* **ternary** *operator*
- *The conditional expression* `expr1 ? expr2 : expr3` *should be read*
  - *"* `if` *expr1*
    - `then` *expr2*
    - `else` *expr3."*
- *The expression is evaluated in stages:*
  - `expr1` *is evaluated first;*
    - *if* `expr1` $\neq 0$, *then* `expr2` *is evaluated,*
      - *and* `expr2` *'s value is the value of the entire conditional expression.*
    - *if* `expr1` $= 0$, *then* `expr3` *is evaluated*
      - *then the value of* `expr3` *is the value of the entire conditional expression.*

```c
int i = 1;
int j = 2;
int k =  i >  j ? i : j;        // k = 2
    k = (i >= 0 ? i : 0) + j;   // k = 3
```

- *The* **parentheses** *are* **necessary**
- *without them*

```c
k =  i >= 0  ?  i  :  0 + j;
k = (i >= 0) ? (i) : (0 + j);
```

  - *this is what happens.*
    - *k = 1*

---

ivan's @ page28

- *The parentheses are necessary*
  - *because the precedence of the* **conditional/ternary** *operator* $<$ *that of the other operators discussed so far,*
    - *with the exception of the* **assignment** *operators*
- **Conditional/Ternary** *Operator*
  - *Makes program shorter, but harder to* **understand**
  - *are often used in* `return` *statements:*

```c
return i > j ? i : j;
```

ivan's @ page30

- *Calls of printf can sometimes benefit from condition expressions.*

```c
if (i > j)
    printf("%d\n", i);
else
    printf("%d\n", j)
```

```c
printf("%d\n", i > j ? i : j);
```

- **Conditional/Ternary** *Operators are also common in certain kinds of* **preprocessor** *macro definitions in large software codebases.*
  - *Chapter 14.3*

5.3.1. The `switch` Statement

A cascaded if statement can be used to compare an expression against a series of values:

```c
if (grade == 3)
    printf("Excellent");
else if (grade == 2)
    printf("Average");
else if (grade == 1)
    printf("Poor");
else if (grade == 0)
    printf("Failing");
else
    printf("Illegal grade");
```

```c
switch (grade) {
    case 3: printf("Excellent");
        break;
    case 2: printf("Average");
        break;
    case 1: printf("Poor");
        break;
    case 0: printf("Failing");
        break;
    default: printf("Illegal grade");
        break;
}
```

- *The* `switch` *statement*
  - *is an alternative*
  - *may be easier to read than a* **cascaded** `if` *statement.*
  - *are often faster than* `if` *statements.*
    - *depends on the compiler.*
- *Remember to* `break` *in each* `case`

Most common form of the `switch` **statement:**

```c
switch ( ctrl-expression ) {
    case constant : statements
    …
    case constant : statements
    default : statements
}
```

- *The word* `switch` *must be followed by the* `ctrl` *-expression*
  - `ctrl` *-expression* **must be** `int`
    - *Characters are treated as integers in* C *and thus can be tested in* `switch` *statements*
    - *However,* `float` *and* `string` *don't qualify*

Each `case` **begins with a label of the form**

```c
case constant-expression :
    statements;
    break;
```

- *A* **constant-expression** *is much like an ordinary expression except that it can't contain variables or function calls*
  - `5` *is a constant expression, and*
  - `5 + 10` *is a constant expression,*
  - *but* `n + 10` *isn't a constant expression*
    - *(unless n is a* **preprocessor macro** *that represents a constant)*

- *A* **constant-expression** *must evaluate to an* `int`
  - `char` *is acceptable*
  - `float` `string` *are not*

Several `case` **labels may precede a group of statements:**

```c
switch (grade) {
    case 3:
    case 2:
    case 1:     printf("Passing");
                break;
    case 0:     printf("Failing");
                break;
    default:    printf("Illegal grade");
                break;
}
```

- *After each* **case** *label comes any number of statements*
- *No* **braces** *are required around the statements*
- *The last statement in each group is normally* `break`

To save space, several `case` **labels can be put on the same line:**

```c
switch (grade) {

    case 3: case 2: case 1:
                printf("Passing");
                break;
    case 0:     printf("Failing");
                break;
    default:    printf("Illegal grade");
                break;
}
```

- *Duplicate* **case** *labels aren't allowed*
- *the order of* **cases** *doesn't matter*
- `default` **case** *doesn't need to come last*

- *If the* `default case` *is* **missing** *and the* `ctrl-expression` $=$ `false`
  - *then nothing happens and we simply pass on to the next statement after the entire big* `switch` *-block*

5.3.2. The `break` Statement 🟩

Now, let's take a closer look at the mysterious break statement. As we've seen, executing a break statement causes the program to "break" out of the switch statement; execution continues at the next statement after the entire big `switch` -block.

The reason that we need break has to do with the fact that the switch statement is really a form of "computed jump." When the `ctrl-expression` is evaluated, control/processor jumps to the case label matching the value of the switch expression.
A case label is nothing more than a marker indicating a position within the switch.

A case label is nothing more than a bookmark

```c
switch (grade) {
    case 4:     printf("Excellent");
    case 3:     printf("Good");
    case 2:     printf("Average");
    case 1:     printf("Poor");
    case 0:     printf("Failing");
    default:    printf("Illegal grade");
}
```

- **If** *grade = 3,*
- **Then** *this is printed*

```
"GoodAveragePoorFailingIllegal grade"
```

  - *Without* **break**, *processor will flow from one case into the next.*

Why?

- **If** *grade = 3*
- **Then** `switch` *sends the processor to* `case 3`
- **But**
  - *The thing is, magically after that, all the* `case 2` `case 1` `case 0` `default` **labels** *vanishes magically into thin air* 🤦
  - *From that point onward, every statement downward is executed.*

🟩 textbook's @ page88

- *When the last statement in the* `case 3` *has been executed,*
  - *control/***processor "falls through"** *next case,*
    - *i.e. get's into the next downwards case*
  - *control/***processor** *starts executing from the first statement of the next downwards case*
  - *the case labels of all the next downwards cases are just ignored.*

🟩 ivan's @ page41

- *Without* **break** *(or some other* **jump** *statement), control will flow from one case into the next.*
- *Omitting* **break** *is sometimes done intentionally,*
  - *but it's usually just an oversight.*

```c
switch (grade) {
    case 4: case 3: case 2: case 1:
            num_passing++;
            /* FALL THROUGH */
    case 0: total_grades++;
        break;
}
```

  - *It's a good idea to point out deliberate omissions of* **break** *using* **comments**
  - *Without the* **comment**,
    - *someone might* **later** *"fix the error"*
      - *by adding an unwanted* **break** *statement @ the place of the* **comment**

5.3.3. Program : `Printing a Date in Legal Form` 🟩

Contracts and other legal documents are often dated in the following way:

```
Dated this __________ day of __________, 20__.
```

- *Let's write a program that displays dates in this form.*

- *We'll have the user enter the date in month/day/year form, then we'll display the date in* **"legal"** *form*
- *Let's say that a broker charges the amounts shown in the following table:*

```
Enter date (mm/dd/yy): 7/19/14
Dated this 19th day of July, 2014.
```

  - *This is just a really dumb task....*
    - 🟦 *Says REYNEP*
  - **Answer on Next page**

  - *We can get* `printf()` *to do most of the formatting.*
  - *However, we're left with two problems:*
    a. how to add **"th"** *(or* **"st"** *or* **"nd"** *or* **"rd"***) to the day, and*
    b. how to print the month as a word instead of a number.
  - *Fortunately, the* **switch** *statement is ideal for solving both our problems;*
    - *we'll have one switch print the day suffix and another print the month name.*

5.3.3. Program : `Printing a Date in Legal Form` ANSWER

```c
/* Prints a date in legal form */
#include <stdio.h>

int main(void) {
    int month, day, year;

    printf("Enter date (mm/dd/yy): ");
    scanf("%d /%d /%d", &month, &day, &year);

    printf("Dated this %d", day);


    switch (day) {
        case 1: case 21: case 31:   printf("st"); break;
        case 2: case 22:            printf("nd"); break;
        case 3: case 23:            printf("rd"); break;
        default:                    printf("th"); break;
    }

    printf(" day of ");


    switch (month) {
        case 1:  printf("January");   break;
        case 2:  printf("February");  break;
        case 3:  printf("March");     break;
        case 4:  printf("April");     break;
        case 5:  printf("May");       break;
        case 6:  printf("June");      break;
        case 7:  printf("July");      break;
        case 8:  printf("August");    break;
        case 9:  printf("September"); break;
        case 10: printf("October");   break;
        case 11: printf("November");  break;
        case 12: printf("December");  break;
    }

    printf(", 20%.2d.\n", year);

    return 0;
}
```

- *Note the use of* `%.2d`
  to display the last two digits of the year.
- *If we had used* `%d` *instead,*
  - *single-digit years would be displayed incorrectly*
    - *(* `2005` *would be printed as* `205` *).*